

2026 年 5 月 13 日 Version 1.0

指揮者人形による自律同期 Arduino オーケストラ

グループ 32

25G1033 : 長見 東磨
25G1098 : 長野 志哉
25G1099 : 長山 魁良
25G1094 : 中村 海惺
24G1011 : 飛田 啓斗
24G1056 : 慶田 優
24G1035 : 尾添 遼太

第 1 章 プロジェクトの計画書

1.1 プロジェクトの概要

1.1.1 プロジェクトの題目

指揮者人形による自律同期 Arduino オーケストラ

1.1.2 プロジェクトの目的

音楽の自動演奏システムは数多く存在するが、その大半は事前に定義された楽譜とテンポを機械的に再生するに留まり、演奏中の人間的な表現が介在する余地は乏しい。とりわけ、指揮者がリアルタイムに与えるアゴーギク、*accelerando*（加速）、*ritardando*（減速）、*Feltemato*（音の引き伸ばし）といったテンポと時間の揺らぎは、音楽の重要な要素の一つである。Borchers らが *Personal Orchestra* として実装した研究 [1] 以降、指揮動作を入力として演奏速度を変更するシステムは複数提案されてきたが、その多くは中央のサーバが指揮を解釈して各音源に同期信号を配信する集中型アーキテクチャを採用しており、人間のオーケストラに見られる「各奏者が指揮者を観測して同期する」という分散的構造とは異なる。

本プロジェクトの目的は、指揮者ロボットの物理的動作を各楽器ユニットが独立に観測し、その指揮表現にリアルタイムに追従して合奏する電子アンサンブルシステムを構築することである。具体的には、サーボモータで反射板を駆動して指揮動作を再現する親機と、赤外線距離センサおよびフォトランジスタで親機を観測する複数の楽器ユニットからなる構成をとり、ユニット間通信を介さない物理的観測のみで同期演奏を実現する。最終的なゴールは、*accelerando*、*ritardando*、*Feltemato* といった指揮の表現変化に対し、各楽器ユニットがそれぞれ追従して合奏できる状態を達成することである。

本プロジェクトにおいて解決すべき技術的課題は、以下の二点である。まず、単一のセンサでは拍タイミングの精度と動作変化の連続観測を両立できないた

め、フォトランジスタによる発光検出と赤外線センサによる距離測定を統合する設計が必要となる。次に、過去の拍履歴から滑らかなテンポ追従を行いつつ、*accelerando/ritardando* による急な変化やフェルマータによる時間の停止を遅滞なく検出する必要がある。

1.1.3 既存の技術とプロジェクトの独自性

1.1.3.1 既存の技術

機械による自動演奏および指揮・演奏への追従に関する研究は、大きく以下の二系統に分類される。

楽譜追従システム 人間奏者の演奏音響をリアルタイム解析し、楽譜上の現在位置を推定して伴奏を行うシステムである。Cont が IRCAM で開発した Antescofo[2] はその代表例であり、確率的推論によって奏者の演奏位置と速度を追跡しながら、電子音響パートを同期させる。これらのシステムは演奏に対して高度に追従するが、追従対象は奏者の音響であり、指揮者の動作ではない。

指揮認識システム カメラやモーションセンサで指揮者の動きを取得し、テンポや音量に変換するシステムである。Borchers らの Personal Orchestra[1] は赤外線バトンを用いた指揮動作認識により、録音をリアルタイムに伸縮させて再生する。本系統の研究の多くは、中央サーバが指揮を解釈し、その結果を音源へ配信する集中型アーキテクチャを採用する。

1.1.3.2 プロジェクトの独自性

上記の既存研究を踏まえ、本プロジェクトは以下の二点に独自性を持つ。

指揮の表現を物理的に直接観測する設計 従来の指揮認識システムがカメラ画像や専用バトンの信号を介して指揮を解釈するのに対し、本プロジェクトでは各楽器ユニットが指揮者ロボットを赤外線センサとフォトランジスタで直接観測する。これにより、振りの速度変化 (*accel./rit.*) と最下点での停滞 (フェルマータ) という指揮者の表現情報を、加工を経ない一次情報として各楽器が受け取る。

2 系統のセンサ統合による拍検出の頑健化 赤外線距離センサによる連続的な腕位置観測と、フォトランジスタによる発光検出という二つの独立した観測系を組み合わせる。後者は時間分解能に優れ拍の瞬間を高精度に捉えられる一方、前者は最下点予測やフェルマータ検出といった連続的な動作状態の判定に適している。両者を指数

移動平均 (Exponential Moving Average; EMA) に基づくテンポ推定器で統合することで、単一センサでは困難な「拍タイミングの精度」と「テンポ変化への追従」の両立を図る。

1.2 スコープ・マネジメント

1.2.1 成果物

本プロジェクトで最終的に完成・納品する成果物を以下に示す。

ハードウェア ・指揮者人形ユニット (サーボ機構・腕・反射板)

- ・指揮者回路 (LED 駆動回路・サーボ駆動回路)
- ・フォトトランジスタ配線・遮光チャンバー

ソフトウェア ・指揮者 Arduino SW (サーボ制御・輝度エンコード LED 制御)

- ・観測 Arduino SW (センサー ISR・EMA テンポ推定・状態機械)
- ・演奏 Arduino SW × 5 (輝度キュー待受・楽譜進行・シリアル送信)
- ・楽譜配列データ (「かえるのうた」5 パート分)
- ・Processing 音響合成 SW (シリアル受信・倍音合成・ADSR・エフェクト)
- ・音色定義データ・ADSR パラメータシート

ドキュメント ・マネジメント計画書 (本文書)

- ・システム基本設計書 (第 2 章)
- ・システム詳細設計書 (第 3 章)

1.2.2 プロジェクトの対象範囲と非対象範囲

1.2.2.1 対象範囲

本プロジェクトが取り組む範囲を以下に示す。

- ・指揮者人形ユニットの設計・製作 (機構・回路・ソフトウェア)
- ・赤外線センサおよびフォトトランジスタによる拍タイミング検出
- ・指数移動平均 (EMA) を用いたリアルタイムテンポ推定
- ・「かえるのうた」5 パート輪唱の同期演奏制御

- ・ テンポ変化（accelerando/ritardando）およびフェルマータへのリアルタイム追従
- ・ Processing による音響合成およびスピーカへの出力

1.2.2.2 非対象範囲

以下は本プロジェクトの対象外とする。

- ・ 「かえるのうた」以外の楽曲への対応
- ・ Arduino ユニット間のワイヤレス（無線）通信
- ・ 演奏中のリアルタイム楽曲切り替え機能
- ・ スマートフォンや Web ブラウザからの外部制御インターフェース
- ・ 製品化・量産を前提とした耐久性設計

1.2.3 機能分解構造 FBS

機能分解構造（FBS：Function Breakdown Structure）は、システムが果たすべき機能を階層的に分解したものである。本プロジェクトの FBS を図 1.1 に示す。

最上位（L0）の「Arduino オーケストラシステム」を、5 つの主機能グループ（L1）に分解する。

- ・ **指揮動作検出**：フォトトランジスタと赤外線センサにより拍タイミングを検出し、EMA でテンポを推定する。フェルマータや見逃し補完も含む。
- ・ **指揮者人形動作**：サーボモータで腕を往復させてテンポを物理提示し、最下点で LED を点灯させることで拍の同期基準を与える。
- ・ **演奏制御**：親機からの輝度キューによる演奏開始制御、楽譜進行、テンポ追従演奏の 3 機能を担う。
- ・ **音響合成**：倍音・ADSR による音色合成、エフェクト付与、フェルマータ時の延奏を行う。
- ・ **状態管理**：待機・予備拍・演奏中・フェルマータ等の状態遷移を管理し、異常時のフォールバックを提供する。

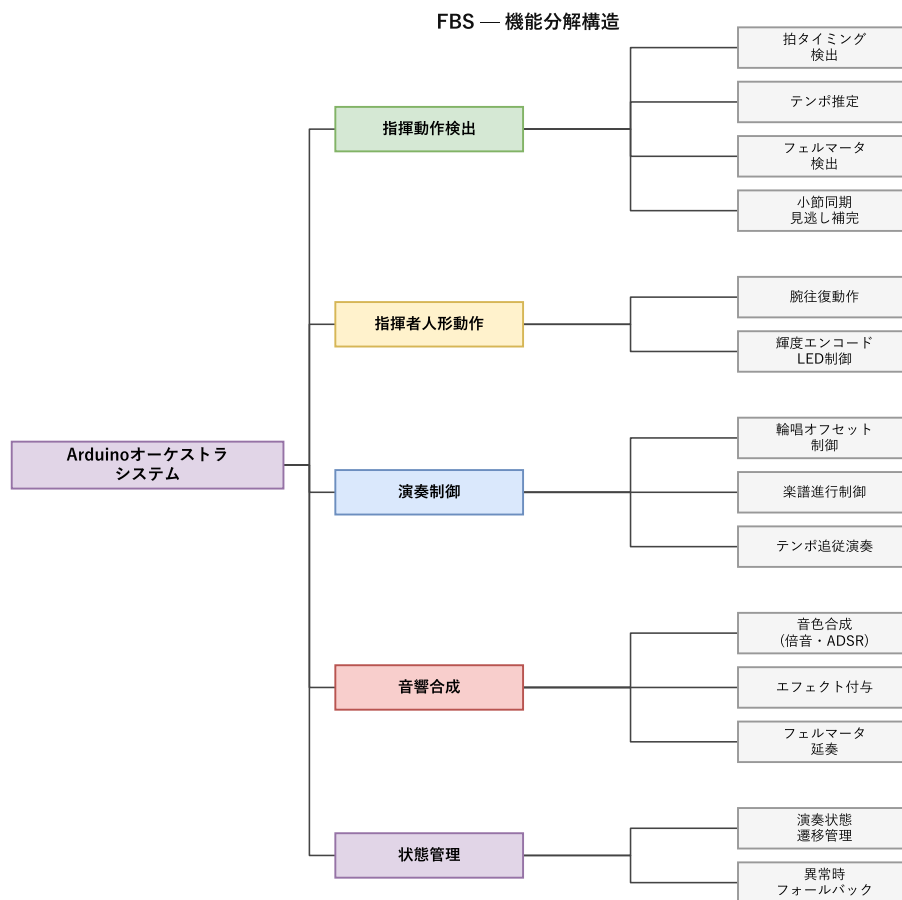


図 1.1 機能分解構造 (FBS)

1.2.4 製品分解構造 PBS

製品分解構造 (PBS : Product Breakdown Structure) は、システムを構成する成果物 (ハードウェア・ソフトウェア・データ) を階層的に分解したものである。本プロジェクトの PBS を図 1.2 に示す。

最上位 (L0) を担当単位 (L1) で分割し、各担当の成果物 (L2) およびその構成要素 (L3) まで展開する。

- ・ 担当 A (指揮者人形ユニット) : サーボ機構・腕・反射板からなる人形機構,

LED・サーボ駆動回路, およびサーボ制御・輝度エンコード LED 制御ソフトウェア, および楽器指定・BPM 入力・演奏制御ボタンを備えた Processing 専用 GUI インターフェース.

- ・ **担当 B (センサー・観測)** : フォトトランジスタ配線 (遮光チャンバー含む), センサー ISR・EMA テンポ推定・状態機械・輝度キュー判別を実装した観測 Arduino SW.
- ・ **担当 C (演奏制御)** : 輝度キュー受信・楽譜進行・シリアル送信を実装した演奏 Arduino SW, および「かえるのうた」5 パート分の楽譜配列データ.
- ・ **担当 D (音響合成)** : シリアル受信・音色合成エンジン・エフェクト処理を実装した Processing SW, および音色定義シートと ADSR パラメータからなる音色定義データ.

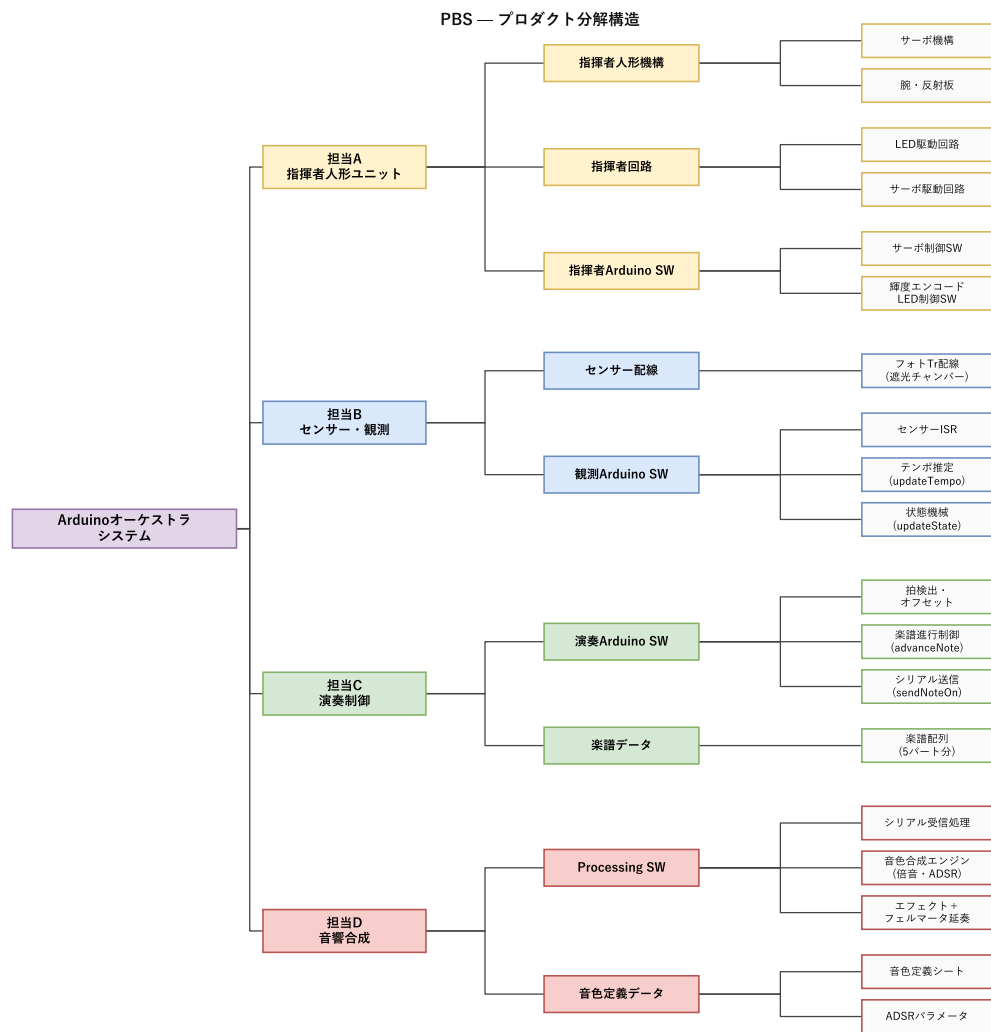


図 1.2 製品分解構造 (PBS)

1.2.5 FBS と PBS の対応関係

FBS の各機能と PBS の各成果物の対応関係を表 1.1 に示す。FBS は「何をするか」、PBS は「何を作るか」を表しており、それぞれの機能は一つ以上の成果物によって実現される。

表 1.1 FBS 機能と PBS 成果物の対応

FBS 機能 (L2)	対応する PBS 成果物 (L2/L3)
拍タイミング検出	センサー配線 (フォト Tr 配線), 観測 Arduino SW (センサー ISR)
テンポ推定	観測 Arduino SW (updateTempo)
フェルマータ検出	観測 Arduino SW (状態機械)
小節同期・見逃し補完	観測 Arduino SW (状態機械)
腕往復動作	指揮者人形機構 (サーボ機構・腕), 指揮者回路 (サーボ駆動回路), 指揮者 Arduino SW (サーボ制御 SW)
輝度エンコード LED 制御	指揮者回路 (LED 駆動回路), 指揮者 Arduino SW (輝度エンコード LED 制御 SW)
演奏開始キュー制御	観測 Arduino SW (輝度判別・開始フラグ管理)
楽譜進行制御	演奏 Arduino SW (advanceNote), 楽譜データ (楽譜配列)
テンポ追従演奏	演奏 Arduino SW (sendNoteOn)
音色合成	Processing SW (音色合成エンジン), 音色定義データ (音色定義シート・ADSR パラメータ)
エフェクト付与	Processing SW (エフェクト+フェルマータ延奏)
フェルマータ延奏	Processing SW (エフェクト+フェルマータ延奏)
演奏状態遷移管理	観測 Arduino SW (状態機械), Processing SW (シリアル受信処理)
異常時フォールバック	観測 Arduino SW (状態機械)

1.3 作業計画

1.3.1 作業分解構造 WBS と管理番号

フェーズ 100（設計）、200（実装）、300（テスト）の 3 フェーズで構成される WBS を図 1.3 に示す。各フェーズには複数の要素成果物と活動タスクが含まれ、それぞれに管理番号と担当者が割り当てられている。

フェーズID	フェーズ名	成果物ID	成果物名	タスクID	タスク名	担当者
100	設計フェーズ	110	共通設計	111	FBS作成	全員
				112	PBS作成	
				113	WBS作成	
				114	MOE・評価基準確定	
		120	担当A設計（指揮者人形ユニット）	121	サーボ機構設計	長見・飛田
				122	腕・反射板設計	
				123	LED駆動回路設計	
				124	サーボ駆動回路設計	
				125	サーボ制御SW仕様書	
				126	輝度エンコードLED制御SW仕様書	
		130	担当B設計（センサー・観測）	131	赤外線センサー配線設計	長野・慶田
				132	フォトTr配線設計	
				133	センサーISR設計	
				134	テンポ推定アルゴリズム設計	
				135	状態機械設計	
		140	担当C設計（演奏制御）	141	拍検出・オフセット設計	長山・慶田
				142	楽譜進行制御設計	
				143	シリアル送信プロトコル設計	
				144	楽譜配列定義・5パート	
		150	担当D設計（音響合成）	151	シリアル受信処理設計	中村・尾添
				152	音色合成エンジン設計	
				153	エフェクト・フェルマータ延奏設計	
				154	音色定義シート作成	
				155	ADSRパラメータ仕様策定	
200	実装フェーズ	210	担当A実装（指揮者人形ユニット）	211	サーボ機構組み立て	長見・飛田
				212	腕・反射板取り付け	
				213	LED駆動回路配線	
				214	サーボ駆動回路配線	
				215	サーボ制御SW実装	
				216	輝度エンコードLED制御SW実装	
		220	担当B実装（センサー・観測）	221	赤外線センサー配線	長野・慶田
				222	フォトTr配線・遮光チャンバー	
				223	センサーISR実装	
				224	テンポ推定実装	
				225	状態機械実装	
		230	担当C実装（演奏制御）	231	拍検出・オフセット実装	長山・慶田
				232	楽譜進行制御実装	
				233	シリアル送信実装	
				234	楽譜配列作成・5パート	
		240	担当D実装（音響合成）	241	シリアル受信処理実装	中村・尾添
				242	音色合成エンジン実装	
				243	エフェクト・フェルマータ延奏実装	
				244	音色定義データ作成	
				245	ADSRパラメータ調整	
300	テストフェーズ	310	事前確認テスト（P1-P6）	311	赤外線センサー精度確認	長野・慶田
				312	フォトTr検出距離確認	長見・飛田
				313	LED輝度・フォトTr検出確認	長野・慶田
				314	サーボ動作角度確認	長見・飛田
				315	サーボ最大BPM確認	
				316	Arduino-Processing通信確認	長野・長山・慶田/中村・尾添
		320	単体テスト（U1-U4）	321	センサーISR・テンポ推定単体テスト	長野・慶田
				322	状態機械単体テスト	
				323	楽譜進行・シリアル送信単体テスト	長山・慶田
				324	音色合成・エフェクト単体テスト	中村・尾添
		330	結合テスト（C1-C3）	331	Arduino-Processing遅延測定	長野・長山・慶田/中村・尾添
				332	同期・テンポ追従確認	長野・長山・慶田
				333	フェルマータ動作確認	長野・長山・慶田/中村・尾添
		340	システムテスト（S1-S3）	341	通し演奏テスト	全員
				342	拍タイミング精度評価	
				343	テンポ変化・フェルマータ完全確認	

図 1.3 作業分解構造（WBS）

1.3.2 アローダイアグラムとクリティカルパス

アローダイアグラム（Activity on Arrow 形式）を図 1.4 に示す。各ノードには最早開始日（ES）と最遅開始日（LS）を記載し、ES = LS となる活動をクリティカルパス（赤矢印）として示す。

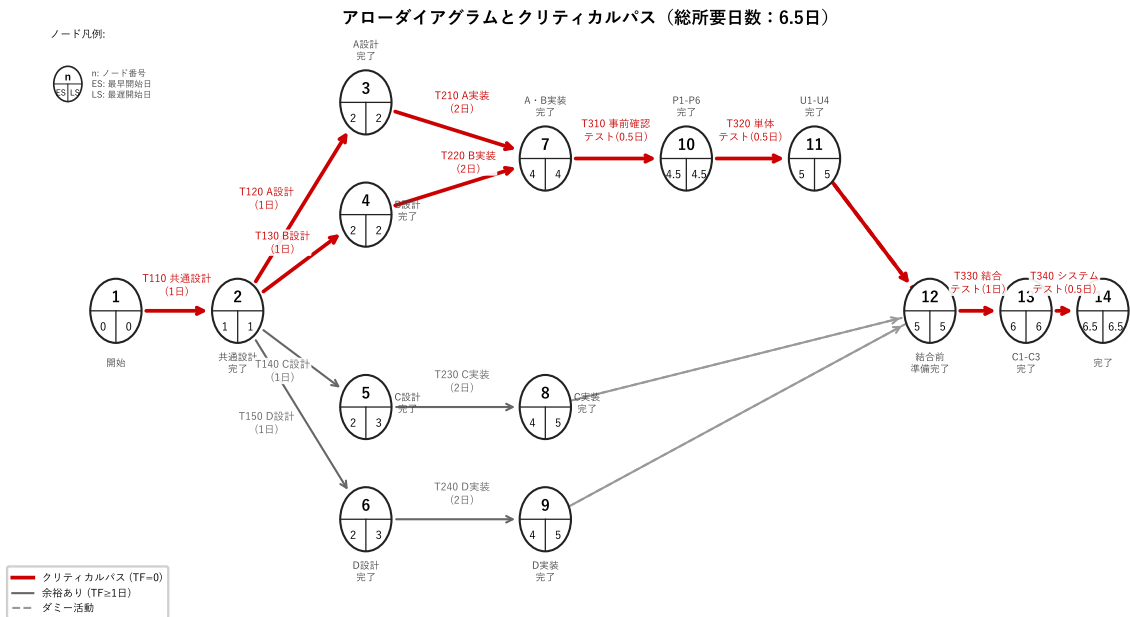


図 1.4 アローダイアグラムとクリティカルパス

クリティカルパスは① → ② → ③ → ⑦ → ⑩ → ⑪ → ⑫ → ⑬ → ⑭（担当 A 実装経路）および① → ② → ④ → ⑦ → ⑩ → ⑪ → ⑫ → ⑬ → ⑭（担当 B 実装経路）の 2 経路であり、総所要日数は **6.5 日**となる。担当 C・D（T140・T150・T230・T240）は各 1 日のトータルフロートを有する。

1.3.3 役割分担

WBS（図 1.3）および PBS に基づき、各機能およびコンポーネントの設計・実装・テストは以下の担当者に割り当てられている。

- ・ 担当 A（指揮者人形ユニット）：長見・飛田
- ・ 担当 B（センサー・観測）：長野・慶田

- ・ 担当 C (演奏制御) : 長山・慶田
- ・ 担当 D (音響合成) : 中村・尾添

全体に関わる共通設計 (WBS 110) や事前確認テスト以降の結合・システムテスト (WBS 300 番台の一部) は、全員または関係する担当者が共同で実施する。

1.3.4 ガントチャート

図 1.5 にガントチャートを示す。フェーズ 200 (実装) は第 6 週 (5 月 18 日) から各担当が並行して着手する。第 7 週 (5 月 25 日) は大学のテスト期間に伴うバッファ一週 (薄橙色) として確保し、実装作業を一時中断する。フェーズ 300 (テスト) は第 9 週 (6 月 8 日) の事前確認テスト (P1-P6) から開始し、単体テスト (U1-U4)、結合テスト (C1-C3) を経て、第 11 週 (6 月 22 日) のシステムテスト (S1-S3) で完了する。

タスクID	タスク名	担当者	第6週 5/18	バッファ 5/25	第6週 6/1	第9週 6/8	第10週 6/15	第11週 6/22
実装フェーズ								
担当A実装（指揮者人形ユニット）								
211	サーボ機構組み立て	長見・飛田						
212	腕・反射板取り付け	長見・飛田						
213	LED駆動回路配線	長見・飛田						
214	サーボ駆動回路配線	長見・飛田						
215	サーボ制御SW実装	長見・飛田						
216	角度エンコード LED制御SW実装	長見・飛田						
担当B実装（センサー・観測）								
221	赤外線センサー配線	長野・慶田						
222	フォトTr配線・遮光チャンバー	長野・慶田						
223	センサーISR実装	長野・慶田						
224	デンプ推定実装	長野・慶田						
225	状態機械実装	長野・慶田						
担当C実装（演奏制御）								
231	拍検出・オフセット実装	長山・慶田						
232	楽譜進行制御実装	長山・慶田						
233	シリアル送信実装	長山・慶田						
234	楽譜配列作成・5パート	長山・慶田						
担当D実装（音響合成）								
241	シリアル受信処理実装	中村・尾添						
242	音色合成エンジン実装	中村・尾添						
243	エフェクト・フェルマータ 延奏実装	中村・尾添						
244	音色定義データ作成	中村・尾添						
245	ADSRパラメータ調整	中村・尾添						

図 1.5 ガントチャート（実装フェーズ）

タスクID	タスク名	担当者	第6週 5/18	バッファ 5/25	第6週 6/1	第9週 6/8	第10週 6/15	第11週 6/22
テストフェーズ								
事前確認テスト (P1-P6)								
311	赤外線センサー精度確認	長野・慶田						
312	フォトTr検出距離確認	長見/長野・慶田						
313	LED輝度・フォトTr検出確認	長野・慶田						
314	サーボ動作角度確認	長見・飛田						
315	サーボ最大BPM確認	長見・飛田						
316	Arduino-Processing通信確認	長野/長山/中村他						
単体テスト (U1-U4)								
321	センサーISR・テンゴ 推定単体テスト	長野・慶田						
322	状態機械単体テスト	長野・慶田						
323	楽譜進行・シリアル 送信単体テスト	長山・慶田						
324	音色合成・エフェクト 単体テスト	中村・尾添						
結合テスト (C1-C3)								
331	Arduino-Processing遅延測定	長野/長山/中村他						
332	同期・テンゴ追従確認	長野/長山・慶田						
333	フェルマータ動作確認	長野/長山/中村他						
システムテスト (S1-S3)								
341	通し演奏テスト	全員						
342	拍タイミング精度評価	全員						
343	テンゴ変化・フェルマータ 完全確認	全員						

図 1.6 ガントチャート（テストフェーズ）

1.4 検証と妥当性確認：V&V 計画

ハッカソン 2 テキスト §1.2.5 に基づき、MOE → MOP → TPM の順で評価指標を定める。実運用では順序が逆転し、実装中に TPM を継続監視し、V&V によって MOP を確認し、最終的に MOE で受入判定を行う。本節では各指標の定義・数値目

標・設定根拠を示す。各 MOP はスコープ・マネジメント節 (§1.2) で定めた FBS の機能要素に対応して設定している。各 MOP の具体的なテスト手順・判定基準は、第 3 章 (システムの詳細設計) の各担当節に設けたテスト設計 (§3.1~§3.4) に記述する。

1.4.1 プロジェクトの成果物に対する有効指標 MOE

MOE (Measure of Effectiveness : 有効指標) は、ステークホルダ (審査員・聴衆) が成果物の合格・不合格を判定するための指標であり、受入テストの合否基準に相当する。本プロジェクトの MOE を表 1.2 に示す。

表 1.2 MOE 一覧

No.	MOE	内容	確認方法
MOE-1	演奏完走	「かえるのうた」5 パート輪唱 (テンポ変化・フェルマータ含む) を最初から最後まで演奏できること	実演で確認
MOE-2	指揮追従	指揮者人形のテンポ変化・フェルマータに対して演奏が連動していることが視聴覚的に確認できること	審査員観察評価
MOE-3	楽器音認識	各パートが意図した楽器 (ピアノ・鉄琴・トランペット・バス/スネア・ハイハット) の音として聴こえること	審査員主観評価

表 1.2 に示すように、MOE は MOE-1 (演奏完走)・MOE-2 (指揮追従)・MOE-3 (楽器音認識) の 3 項目からなる。MOE-1 は本システムの最低限の動作要件であり、演奏が途中で止まることは致命的な失敗にあたるため設定した。MOE-2 は本システムの独自性である「指揮者人形の観測による自律同期」が実際に機能しているかを聴衆・審査員が知覚できることを確認するために設定した。MOE-3 は Processing による音響合成の品質を評価するために設定した。

1.4.2 有効指標 MOE に対応する性能指標 MOP

MOP (Measure of Performance : 性能指標) は MOE を達成するために必要なシステム性能を定量的に示す指標であり、V 字モデルの設計段階で定義する。FBS の各

機能に対応する性能要素に対して設定した MOP と、対応する MOE を表 1.3 に示す。

表 1.3 MOE と MOP の対応

No.	対応 MOE	MOP
1	MOE-1,2	フォトランジスタによる拍検出成功率
2	MOE-1,2	楽器間の発音タイミング誤差
3	MOE-1,2	テンポ変化後の EMA 収束拍数
4	MOE-1,2	腕停止からフェルマータ状態移行までの拍数
5	MOE-3	各パートの発音周波数精度

表 1.3 に示す MOP はそれぞれ MOE-1・MOE-2 の達成に関わる技術的な性能要素であり、各テストフェーズにおいて数値目標として評価する。具体的な目標値はシステムテスト・結合テスト・単体テストの各節で示す。

1.4.3 システムテストで評価する性能指標 MOP

システム全体を統合した状態で評価する MOP を表 1.4 に示す。

表 1.4 システムテストで評価する MOP

No.	対応 FBS 機能	MOP (目標値)
SYS-1	テンポ推定	テンポ変化後の収束： ≤ 3 拍
SYS-2	フェルマータ検出	腕停止からフェルマータ移行： ≤ 1 拍
SYS-3	楽譜進行制御	全区間通し演奏の完走率：100 %

表 1.4 に示す各 MOP の目標値の設定根拠を以下に示す。

SYS-1 の目標値 3 拍の根拠を示す。本楽曲「かえるのうた」は 4/4 拍子（1 小節=4 拍）で演奏される。EMA の更新式は $T_n = \alpha \cdot \Delta t_n + (1 - \alpha) \cdot T_{n-1}$ であり、テンポ変化後の残差は $(1 - \alpha)^n$ で減衰する。テンポ変化を検出した際の平滑化係数 $\alpha = 0.5$ を用いると、 n 拍後の残差は 0.5^n となり、 $n = 3$ のとき $0.5^3 = 0.125$ （87.5%収束）である。4/4 拍子において、テンポ変化後に 1 小節未満（ ≤ 3 拍）で収束すれば、次の小節の先頭から新テンポで各楽器が揃って演奏を再開できる。この「次小節頭で揃う」という音楽的要件と $\alpha = 0.5$ の EMA 特性の両面から、 ≤ 3 拍を目標値とした。

SYS-2 の目標値 1 拍の根拠を示す。フェルマータは楽譜上で特定の拍に付与される記号であり、腕停止を検出してから 1 拍以上経過すると、その拍の音符が余分に発音されて旋律が崩れる。したがって、腕停止検出から次の拍が来る前にフェルマータ状態へ移行しなければならず、この楽譜上の制約から ≤ 1 拍を上限として設定した。フェルマータ検出の具体的な実装は FBS 機能 F2a4（フェルマータ検出）に対応し、詳細は担当 B の詳細設計（第 3 章 §3.2）に記述する。

SYS-3 の 100%は MOE-1（演奏完走）の定義そのものであり、システムテストの合格条件として妥協のない値として設定した。

なお、各システムテストの実施手順・判定基準は第 3 章の各担当詳細設計節に記述する。

1.4.4 結合テストで評価する性能指標 MOP

複数の担当成果物を結合した状態で評価する MOP を表 1.5 に示す。

表 1.5 結合テストで評価する MOP

No.	対応 FBS 機能	MOP（目標値）
INT-1	テンポ追従演奏	楽器間発音タイミング誤差： $\leq \pm 20$ ms
INT-2	テンポ追従演奏	Arduino → Processing 転送遅延： ≤ 50 ms
INT-3	輪唱オフセット制御	輪唱オフセット誤差： $\leq \pm 1$ 拍

表 1.5 に示す各 MOP の設定根拠を以下に示す。INT-1 の目標値 ± 20 ms は、人間が音楽的なタイミングのズレを知覚できる閾値（先行研究では概ね 20 ~ 30 ms[3, 4]）の下限値を採用した。INT-2 の目標値 50 ms の根拠を示す。115200 bps のシリアル通信では、スタート・データ 8bit・ストップビットを含む 1 バイト=10bit を送信する。3 バイトパケットの純粋な転送時間は $\frac{3 \times 10}{115200} \approx 0.26$ ms である。これに Processing 側のバッファ読み取り間隔（典型 10 ms 未満）を加えても実測値は数 ms 程度であり、50 ms は十分な余裕を持った上限値である。また、各楽器ユニットで遅延が一定であれば遅延補正值で吸収できるため、絶対値よりも安定性が重要である。

INT-3 の目標値 ± 1 拍の根拠を示す。輪唱は各パートが一定拍数ずれて同じ旋律を演奏する構造であり、オフセットが ± 1 拍以上ずれると隣接パートの音符と重なり旋律が崩れる。したがって輪唱として成立する最小単位である「1 拍」をそのまま上限として設定した。

なお、結合テストの具体的な実施手順・タイムスタンプ計測方法は、担当 B・C の詳細設計（第 3 章 §3.2・§3.3）に記述する。

1.4.5 単体テストで評価する性能指標 MOP

各担当成果物を単独で評価する単体テストの MOP を表 1.6 に示す。

表 1.6 単体テストで評価する MOP

No.	MOP（目標値）
UNIT-1	拍検出成功率： $\geq 95\%$
UNIT-2	距離計算誤差： $\leq \pm 3\text{ cm}$
UNIT-3	EMA 通常演奏時（ $\alpha = 0.2$ ）の収束： ≤ 13 拍
UNIT-4	NOTE_ON 送信タイミング誤差： $\leq \pm 10\text{ ms}$
UNIT-5	発音周波数精度：目標周波数の $\pm 1\text{ Hz}$ 以内
UNIT-6	BPM 追従誤差： $\leq \pm 5\text{ bpm}$
UNIT-7	ストローク変動： $\leq \pm 2\text{ mm}$

表 1.6 に示す各 MOP の設定根拠を以下に示す。

UNIT-1 の目標値 95% の根拠を示す。検出失敗率を $p = 1 - 0.95 = 0.05$ とすると、3 拍連続で失敗する確率は $p^3 = 0.05^3 = 1.25 \times 10^{-4}$ （約 0.013%）である。フォールバック処理は 3 拍連続未検出で発動するため、検出成功率 95% を維持すればフォールバックがほぼ発動しないことが保証される。

UNIT-2 の目標値 $\pm 3\text{ cm}$ の根拠を示す。フェルマータ検出は絶対距離ではなく腕の「変化量」 Δd で判定するため、絶対距離の精度要件は緩やかである。センサー出力電圧 V から距離 d を推定する近似式 $d = 27.728 \times V^{-1.2045}$ において、Arduino の AD 変換（10bit, 5V 系）の量子化誤差 $\Delta V \approx 5/1024 \approx 4.9\text{ mV}$ を伝搬させると、 $d \approx 30\text{ cm}$ 付近での距離誤差は数 mm～数 cm 程度となる。この誤差範囲を包含する上限として $\pm 3\text{ cm}$ を設定した。

UNIT-3 の目標値 13 拍の根拠を示す。 $\alpha = 0.2$ の EMA が 95% 収束（残差 $\leq 5\%$ ）するのに必要な拍数は、 $(1 - \alpha)^n \leq 0.05$ を解いて $n \geq \frac{\ln 0.05}{\ln(1 - \alpha)} = \frac{\ln 0.05}{\ln 0.8} \approx 13.4$ 拍、切り上げで 14 拍となる。14 拍以内に収束することを確認するため、余裕を 1 拍見て 13 拍を確認目標とした。

UNIT-4 の目標値 $\pm 10 \text{ ms}$ の根拠を示す。楽器間タイミング誤差 (INT-1 : $\pm 20 \text{ ms}$) は、各楽器の送信誤差が独立に重なった合計で生じる。最悪ケースで 2 つの楽器が逆方向に最大誤差を持つと、 $(\pm 10) - (\pm 10) = \pm 20 \text{ ms}$ となる。よって、各楽器単体の誤差を INT-1 の半分である $\pm 10 \text{ ms}$ 以内に抑えることで INT-1 を満たせる。

UNIT-5 の目標値 $\pm 1 \text{ Hz}$ は、音楽的に許容されるピッチのずれ (約 $\pm 5 \text{ cent}$, $A4=440 \text{ Hz}$ で約 $\pm 1.3 \text{ Hz}$ [5]) に対して余裕を持たせた値である。

UNIT-6 の目標値 $\leq \pm 5 \text{ bpm}$ の根拠を示す。80 bpm を基準テンポとすると、1 拍の周期は $60/80 = 750 \text{ ms}$ である。人間がテンポ変化を知覚できる閾値は一般に約 3~6%とされており、80 bpm の 6%は $80 \times 0.06 = 4.8 \text{ bpm}$ である。この値を切り上げた $\pm 5 \text{ bpm}$ を、聴衆が追従と判断できる上限として設定した。

UNIT-7 の目標値 $\leq \pm 2 \text{ mm}$ の根拠を示す。指揮者人形のストローク幅は EMA (担当 B) が計算する「振り幅」に対応し、ストローク変動が大きいと BPM 推定の誤差 (UNIT-6) に直接影響する。UNIT-6 の目標値 $\leq \pm 5 \text{ bpm}$ を 80 bpm における 1 拍周期の変動量に換算すると、 $\Delta T = 60/80 - 60/85 \approx 44 \text{ ms}$ であり、これを引き起こすストローク誤差は機構設計 (担当 A 詳細設計 §3.1) の寸法から $\pm 2 \text{ mm}$ 以内と見積もられる。

各単体テストの具体的な実施手順は、第 3 章の各担当詳細設計節 (担当 A : §3.1, 担当 B : §3.2, 担当 C : §3.3, 担当 D : §3.4) のテスト設計に記述する。

1.4.6 プロジェクト中に監視する技術性能指標 TPM

TPM (Technical Performance Measure : 技術性能指標) は、開発・実装中に継続的に測定・報告する定量的指標であり、技術的リスクの早期発見を目的とする。MOP のうち特にシステム完成時の性能に大きく影響し、実装の早い段階から測定できる項目を選定した。TPM 一覧を表 1.7 に示す。

表 1.7 TPM 一覧

No.	TPM (監視値・許容範囲)
TPM-1	距離計算誤差： $\leq \pm 3 \text{ cm}$
TPM-2	拍検出成功率： $\geq 95 \%$
TPM-3	Arduino → Processing 転送遅延： $\leq 50 \text{ ms}$
TPM-4	EMA ($\alpha = 0.2$) テンポ収束拍数： ≤ 13 拍
TPM-5	指揮者人形が追従できる最大 BPM： $\geq 104 \text{ bpm}$

1.4.7 TPM の監視体制

表 1.8 に、各 TPM の監視方法を示す。監視方法の詳細（使用機材・記録フォーマット）は第 3 章各担当詳細設計節のテスト設計に記述する。

表 1.8 TPM 監視体制

TPM	監視方法
TPM-1	既知距離での実測値と近似式の計算値を比較し誤差を記録する
TPM-2	メトロノームに合わせた指揮動作でフォト Tr の検出率をシリアルモニターで計測する
TPM-3	Arduino 側の送信タイムスタンプと Processing 側の受信タイムスタンプの差分を計測・記録する
TPM-4	既知 BPM のシミュレーションデータを入力して EMA の収束拍数を計測する
TPM-5	指定 BPM で指揮者人形を動作させ、脱調・遅延の有無をシリアルモニターで確認する

1.5 資源・調達管理

1.5.1 人的資源

本プロジェクトの人的資源は7名のメンバーで構成される。各メンバーは担当A～Dのいずれかに所属し、設計・実装・テストの全工程を通じてその担当領域を主体的に推進する（表1.9）。

表 1.9 人的資源一覧

担当	メンバー	主な責務
A（指揮者人形ユニット）	長見・飛田	人形機構組み立て，LED・サーボ駆動回路，サーボ制御 SW
B（センサー・観測）	長野・慶田	センサー配線，センサー ISR・EMA テンポ推定・状態機械 SW
C（演奏制御）	長山・慶田	楽譜配列作成，拍検出・楽譜進行・シリアル送信 SW
D（音響合成）	中村・尾添	音色定義データ，音色合成エンジン・エフェクト Processing SW

共通設計（WBS 110）および結合・システムテスト（WBS 330・340）は全員または関係担当者が共同で実施し、特定担当に過負荷が集中しないよう調整する。

1.5.2 機材・調達計画

本プロジェクトに必要なハードウェア機材を表 1.10 に示す。実験室に常備されている機材（既備）は追加調達不要とし、不足分のみを授業実施機関に申請または自費で調達する。

表 1.10 機材・調達計画

機材名	用途	数量	調達区分
Arduino Uno R4 WiFi	指揮者・観測・演奏各制御	6	既備
サーボモータ (SG92R)	指揮者人形の腕往復動作	1	既備
赤外線距離センサ (Sharp GP2Y0A21YK)	腕位置連続観測	1	既備
フォトランジスタ	指揮者 LED 発光の拍検出	5	既備
高輝度 LED (赤外線)	指揮者側拍通知光源	2	既備
抵抗・コンデンサ類	各回路の定数設定	適量	既備
ブレッドボード・ジャンパ線	プロトタイプ配線	適量	既備
スピーカ (パッシブ)	Processing 音声出力	5	既備
工作材料 (段ボール・スポンジ等)	人形機構・遮光チャンバー筐体	適量	申請調達
PC (Processing 実行用)	音響合成・シリアル受信	6	持ち込み

調達が必要な物品 (工作材料) は第 5 週 (5 月 11 日) までに申請を完了し、第 6 週 (5 月 18 日) の実装開始に間に合うよう手配する。電子部品は実験室常備品で賄えない場合のみ、担当 A が第 5 週中に代替品の確認および申請を行う。

1.5.3 開発環境

本プロジェクトで使用するソフトウェア開発環境を以下に示す。いずれも無償利用可能なツールであり、追加ライセンス費用は発生しない。

- ・ **Arduino IDE 2.x** : Arduino の SW 開発・書き込み (担当 A・B・C)
- ・ **Processing 4.x** : 音響合成 SW の開発・実行 (担当 D)
- ・ **draw.io** : FBS・PBS・WBS 図の作成・管理 (全員)
- ・ **Git / GitHub** : ソースコードのバージョン管理 (全員)
- ・ **LuaLaTeX / jlreq** : 本計画書・設計書のドキュメント作成 (全員)

1.6 リスク管理

1.6.1 リスクの識別と評価

プロジェクト遂行上のリスクを識別し，発生確率（H/M/L）と影響度（H/M/L）の組み合わせでリスクレベルを評価した結果を表 1.11 に示す．

表 1.11 リスク一覧と評価

No.	リスク名	内容	確率	影響
R-1	センサー精度不足	フォト Tr・赤外線センサが要求精度（UNIT-1：95%，UNIT-2：±3 cm）を達成できず，拍検出やフェルマータ検出が不安定になる	M	H
R-2	サーボ最大 BPM 不足	SG90 のトルク・応答速度が不足し，目標最大 BPM（TPM-5：104 bpm）で脱調または動作遅延が生じる	M	M
R-3	EMA チューニング失敗	α の値が適切に調整できず，テンポ収束（SYS-1：≤ 3 拍）やフェルマータ検出（SYS-2：≤ 1 拍）が達成できない	M	H
R-4	通信遅延超過	Arduino → Processing 間のシリアル遅延が INT-2（≤ 50 ms）を超え，発音タイミングがずれる	L	M
R-5	機材調達遅延	3D プリント材料等の申請承認が遅れ，第 6 週の実装開始が遅延する	L	H
R-6	スケジュール圧迫	各担当の実装が第 8 週までに完了せず，第 9 週からのテストフェーズが縮小または中止になる	M	H
R-7	遮光チャンバー性能不足	外乱光によりフォト Tr が誤検出し，拍検出成功率（UNIT-1）が低下する	M	M

1.6.2 リスク対応計画

表 1.11 で識別した各リスクに対する対応策を表 1.12 に示す.

表 1.12 リスク対応計画

No.	リスク名	対応策（軽減/回避/受容）	トリガー条件
R-1	センサー精度不足	【軽減】 事前確認テスト（P1-P3）でセンサー単体精度を第 9 週頭に検証し、不足時はセンサー取り付け位置・遮光調整または閾値再設定を行う	UNIT-1<95%または UNIT-2> ±3 cm
R-2	サーボ最大 BPM 不足	【軽減】 事前確認テスト（P4-P5）でサーボ最大 BPM を実測し、不足時は腕のストローク角度を縮小して応答速度を確保する	TPM-5<104 bpm
R-3	EMA チューニング失敗	【軽減】 シミュレーションデータによる単体テスト（U1）で EMA の収束特性を事前検証し、 α を段階的に調整して目標範囲内に収める	収束拍数 >3 拍またはフェルマータ移行 >1 拍
R-4	通信遅延超過	【受容・軽減】 115200 bps の遅延は理論上 1 ms 未満であり発生確率は低い。万一超過した場合はバッファ読み取り処理を Polling からイベント駆動に変更する	TPM-3>50 ms
R-5	機材調達遅延	【回避】 第 5 週（5 月 11 日）中に申請を完了し、承認が遅れる場合は手持ちの材料で代替した簡易筐体を先行して製作する	第 6 週開始時点で材料未着
R-6	スケジュール圧迫	【軽減】 第 7 週のバッファーク週間を活用して遅延分を吸収する。それでも遅延が残る場合は、未完了タスクをテストフェーズ前半（第 9 週）に繰り越し、テストスコープを縮小して対応する	第 8 週末時点で実装完了率 <90%
R-7	遮光チャンバー性能不足	【軽減】 遮光チャンバーに黒色テープ・フォームを追加して外乱光を遮断する。それでも改善しない場合はフォト Tr をシールドケースに封入する	屋内通常照明下で UNIT-1<95%

1.6.3 リスク監視体制

各担当は週次のミーティング（授業時間内）に TPM の計測結果を報告し，表 1.11 のトリガー条件に抵触したリスクが発生した場合はその場で対応策の着手を決定する．リスクレベルが H×H（高確率かつ高影響）に達した場合は全員でスコープの再確認を行い，MOE-1（演奏完走）の達成を最優先として成果物を取捨選択する．

第 2 章 システムの基本設計

設計書は作成する成果物によって変化します。適宜修正して利用してください。

2.1 担当 A — 指揮者人形・センサ機構

2.1.1 機能一覧

担当 A では、楽器ユニット（子機）が演奏を同期させるための指揮者人形ユニット（親機）を開発を担当する。指揮者人形ユニットは、Arduino オーケストラにおいて演奏の開始・停止の合図を出し、一定のテンポで腕を振ることで、各楽器ユニット（子機）が同期させるための基準となる役割を担う。

本ユニットが担う主な機能は以下の通りである。

- ・ **テンポ提示機能**

腕（反射板）を一定周期で滑らかに往復運動させ、1 往復を 1 拍としてオーケストラ全体のテンポを提示する。

- ・ **拍の同期機能**

腕が最前点に到達した瞬間に、反射板に搭載した LED を短時間点灯させる。子機は「LED の光」と「反射板までの距離の変動」の 2 つの物理的变化を観測することで、通信に頼らず性格に拍を同期する。

- ・ **演奏状態の表現機能**

「待機」「予備拍」「演奏中」「フェルマータ」「演奏停止」の各状態を持ち、状態に応じた動作の変化によって演奏の展開を表現する。

- ・ **外部制御・情報表示機能**

PC 上の制御アプリケーションからシリアル通信経由で制御コマンド（START・STOP・FERMATA）を受信する。また、現在の演奏状態を本体の LCD ディスプレイに表示し、ユニット単体での状況把握を可能にする。

2.1.2 システム構成図

本システムは、ユーザが操作する PC 側の制御アプリケーション (Processing), 制御と状態管理を担うマイクロコントローラ (Arduino Uno R4 WiFi), および動作と状態提示を行う各出力デバイスによって構成される. PC からの制御コマンド受信, FspTimer を用いた割り込み処理, およびマイクロ秒単位でのデバイス同期のフローを含む, システム全体の構成図を以下の図 2.1 に示す.

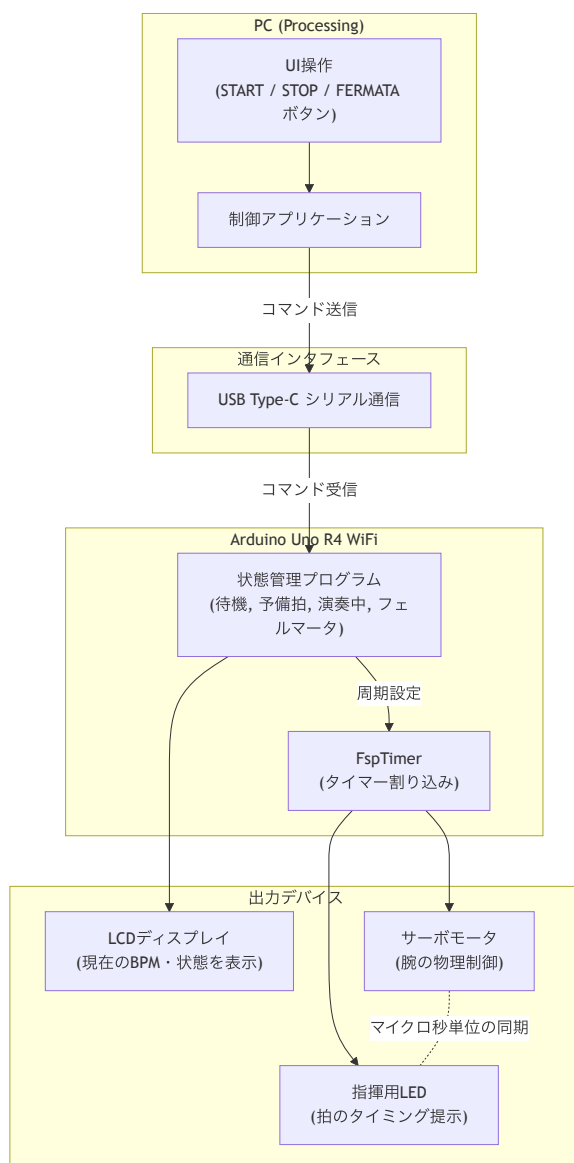


図 2.1 システム構成図

2.1.3 操作インターフェース設計

本システムでは、信号入力用の制御アプリケーション（Processing を用いて構築）を介して指揮者人形ユニットの操作を行う。ユーザインターフェースには、図 2.2 に示すように「START」「STOP」「FERMATA」の各制御ボタンを配置する。ユーザが PC 上でこれらのボタンをクリックすることで、USB Type-C を用いたシリアル通信を介して、対応する信号が Arduino へ送信される。



図 2.2 ユーザーインターフェース表示案

2.1.4 状態遷移図

本システムにおける状態遷移図を以下の図 2.3 に示す。

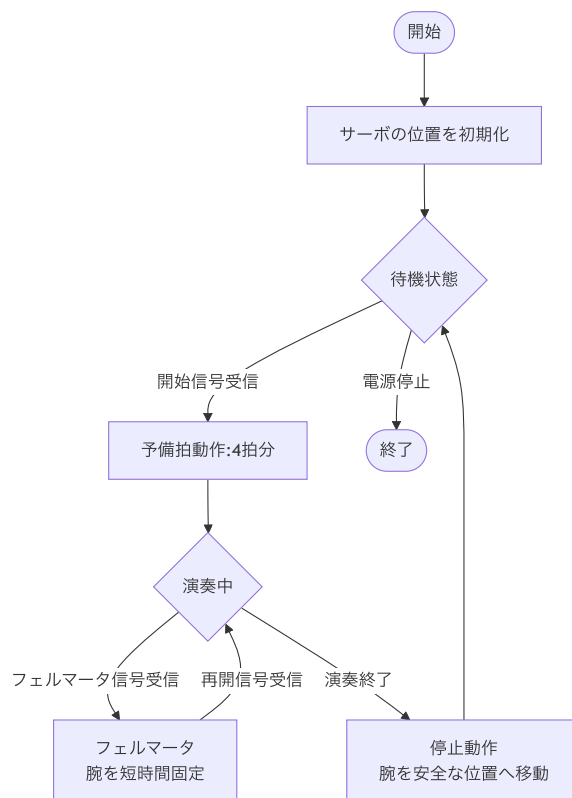


図 2.3 指揮者人形ユニットにおける状態遷移図

2.2 担当 B — センサ処理・状態機械

2.2.1 機能一覧

担当 B では、楽器ユニットにおけるセンサー処理および Arduino による制御を担当する。本システムでは、指揮者ロボットの腕の動きをセンサーによって観測し、その情報をもとに演奏タイミングを決定する。

各楽器ユニットは独立して動作しており、通信を用いることなく指揮者の動きに追従して同期演奏を行う。担当 B はその中核として、動きの検出とタイミング推定を担う。

各楽器ユニットは、あらかじめ内部に楽譜データを保持しており、取得した拍情報に基づいて順次演奏を行う。また、各楽器の演奏開始タイミングは、親機からの輝度

キューによって動的に指示される。演奏者が Processing GUI から任意の順序で各楽器を指定することで、輪唱のような演奏を実現する。

担当 B が提供する主な機能を以下に示す。

- ・ フォトトランジスタによる拍タイミングの検出
- ・ 赤外線センサによる腕の位置・距離の計測
- ・ 移動平均および正規化による距離データの平滑化
- ・ EMA（指数移動平均）によるテンポ推定
- ・ 赤外線センサによる次拍時刻の予測
- ・ テンポ変化の検出と追従
- ・ 腕の静止判定によるフェルマータ検出
- ・ 状態機械による演奏状態の管理
- ・ 担当 C への拍タイミング・状態情報の共有

2.2.2 システム構成図

図 2.4 にシステム全体の構成を示す。

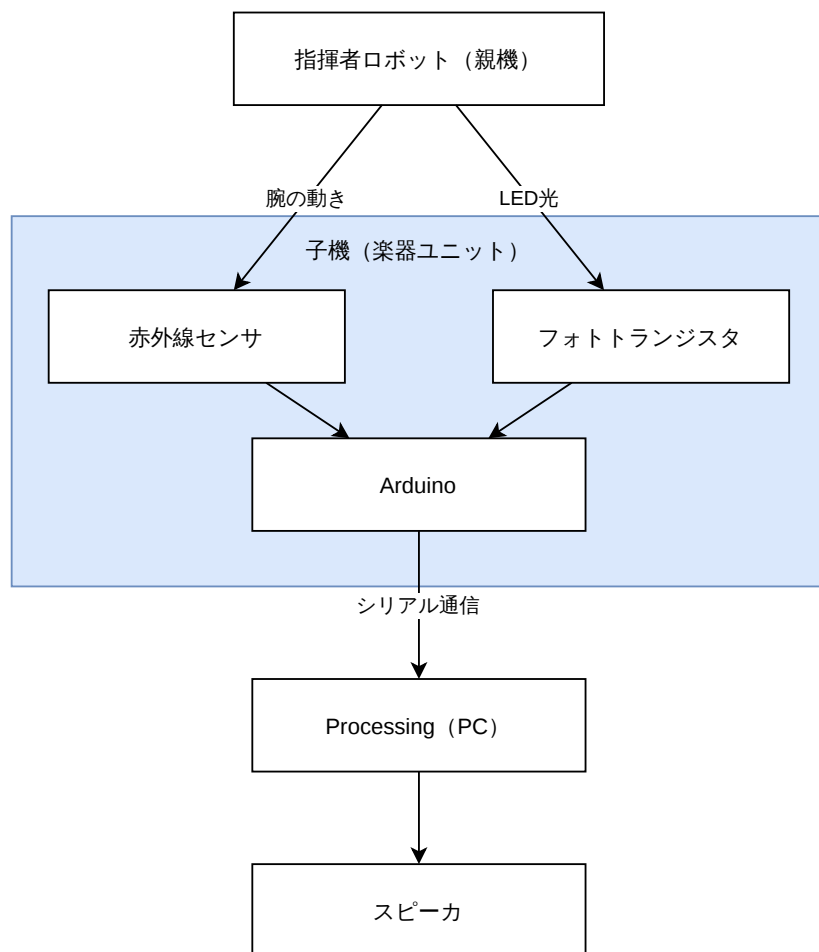


図 2.4 システム構成図

2.2.3 操作インターフェース設計

本システムでは、指揮者ロボットの腕の動きが操作インターフェースとなる。各楽器ユニットはセンサーを通じて指揮者の動きを観測することで、ユーザーが個別の楽器を操作することなく演奏タイミングを制御できる。

センサによるテンポ計測方法を以下に述べる。

計測方法

本システムでは、フォトトランジスタと赤外線センサの2種類のセンサを用いて指揮者の動きを計測する。フォトトランジスタにより拍のタイミングを検出し、赤外線センサにより腕の位置や動きを取得することで、テンポの推定および状態判定を行う。

拍の検出（フォトトランジスタ）

フォトトランジスタは、受光量に応じて電流が変化する素子である。指揮者の腕が振り下ろされた瞬間に点灯するLEDの光を受光することで、出力電圧が変化する。この電圧変化をデジタル入力ピンで検出することにより、拍の基準となるタイミングを高精度に取得する。

腕の位置検出（赤外線センサ）

赤外線センサは、赤外線LEDから照射した光が対象物に反射して返ってくる強度をフォトトランジスタで受光し、その強度から距離を推定する素子である。反射強度は距離の増加に伴い減少するため、出力電圧 V から距離 d [cm] を以下の近似式によって算出する。

$$d = 27.728 \times V^{-1.2045} \quad (2.1)$$

赤外線センサは腕までの距離を一定周期で取得し、腕の位置や動きの変化を連続的に観測する。腕が下がり切った状態は、距離が最小となる点として検出される。

腕の静止判定

腕が静止しているかどうかは、距離の変化が小さい状態が一定回数継続しているかどうかで判定する。

具体的には、距離の移動平均値の変化量があらかじめ設定した閾値以下である状態が複数回連続した場合に、腕が停止していると判断する。また、距離データにはノイズが含まれるため、移動平均を用いて平滑化する。

さらに、腕が振りの最下点や最上点において一時的に速度が低下することによる誤検出を防ぐため、静止状態と判定するには、一定時間以上連続して静止条件を満たすことを必要とする。これにより、瞬間的な停止とフェルマータ状態を区別する。

また、各楽器ユニット間でセンサ値のスケールを統一するため、演奏開始前の予備振り期間中に距離の最大値 $d_{\max}$ および最小値 $d_{\min}$ を記録し、以下の式によって距離を正規化する。

$$d_{\text{norm}} = \frac{d - d_{\min}}{d_{\max} - d_{\min}} \times 100 \quad [\%] \quad (2.2)$$

これにより、静止判定の閾値を全楽器ユニットで統一することができる。

テンポ推定

本システムでは、フォトトランジスタによる LED 検出と赤外線センサによる最下点予測の 2 つの方法を組み合わせでテンポを推定する。

フォトトランジスタによって取得した連続する 2 拍の検出時刻から拍間隔 Δt_n を求め、指数移動平均 (Exponential Moving Average; EMA) によって拍間隔推定値 T_n を更新する。

$$T_n = \alpha \cdot \Delta t_n + (1 - \alpha) \cdot T_{n-1} \quad (2.3)$$

ここで α は EMA 係数 ($0 < \alpha < 1$) であり、値が大きいほど最新の観測値を重視する。通常演奏時は $\alpha = 0.2$ とし、過去の履歴を重視することで急激な変動を抑えた滑らかなテンポ追従を実現する。

次の拍の予測時刻 t_{next} は以下の式で計算する。

$$t_{\text{next}} = t_{\text{beat}} + T_n \quad (2.4)$$

EMA による予測時刻 t_{next} と赤外線センサによる最下点到達予測時刻 t_{predict} を比較し、誤差率が閾値 δ を超えた場合にテンポ変化と判断する。

$$\text{誤差率} = \frac{|t_{\text{next}} - t_{\text{predict}}|}{T_n} \times 100 \quad [\%] \quad (2.5)$$

誤差率が δ (デフォルト 10%) を超えた場合はテンポ変化状態に移行し、赤外線センサの予測値を優先して採用する。誤差率が δ 以内の場合は通常演奏状態を継続し、EMA による予測値を採用する。

本システムでは、指揮者の予備振りとして 4 回の拍を観測した後に、各楽器ユニットは演奏を開始する。また、各楽器は親機からの輝度キューを受信したタイミングで

演奏を開始する。

表 2.1 指揮者の振り回数と各楽器の動作例（輝度キュー受信で開始）

振り回数	指揮者	楽器 1	楽器 2	楽器 3
0	静止	停止	停止	停止
1	振り開始	予備振り	予備振り	予備振り
2	振り	予備振り	予備振り	予備振り
3	振り	予備振り	予備振り	予備振り
4	振り（輝度キュー：楽器 1）	演奏開始	予備振り	予備振り
5	振り（輝度キュー：楽器 2）	演奏中	演奏開始	予備振り
6	振り（輝度キュー：楽器 3）	演奏中	演奏中	演奏開始
7	振り	演奏中	演奏中	演奏中

2.2.4 状態遷移図

本システムでは、演奏状態を複数の状態に分けて管理する。

- ・ 停止状態：演奏を行わない状態
- ・ 予備振り状態：センサの最大・最小値を記録しつつ、テンポを確定するために拍を観測する状態
- ・ 通常演奏状態：EMA による予測値に基づいて演奏を行う状態
- ・ テンポ変化状態：赤外線センサの予測値に基づいてテンポ変化に追従する状態
- ・ フェルマータ状態：音を伸ばす状態

図 2.5 に状態遷移関係を示す。

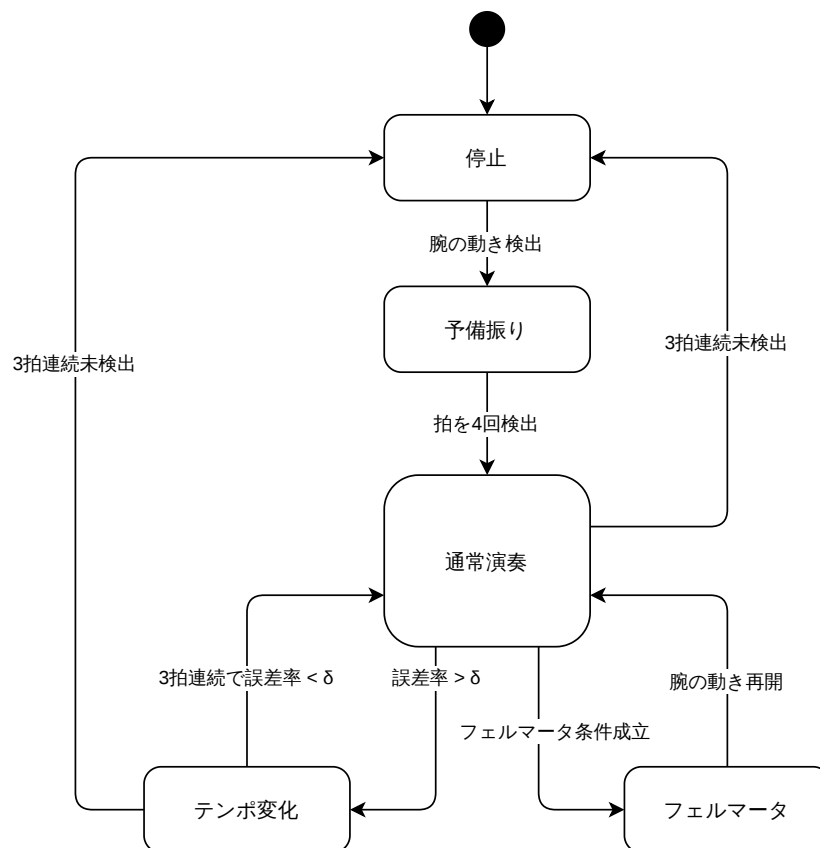


図 2.5 状態遷移図

各状態は、センサーから取得した情報に基づいて以下の条件で遷移する。

- ・ 停止状態 → 予備振り状態
腕の動きが検出された場合に遷移する。
- ・ 予備振り状態 → 通常演奏状態
拍が 4 回検出された場合に遷移する。
- ・ 通常演奏状態 → フェルマータ状態
次の拍の予測時刻を超過し、かつ距離変化が小さい状態が一定時間以上継続した場合に遷移する。
- ・ 通常演奏状態 → テンポ変化状態
EMA による予測時刻と赤外線センサによる予測時刻の誤差率が閾値 δ を超えた場合に遷移する。

- ・ テンポ変化状態 → 通常演奏状態
誤差率が閾値 δ 以内の状態が 3 回連続した場合に遷移する.
- ・ フェルマータ状態 → 通常演奏状態
腕の動きが再び検出された場合に遷移する.
- ・ 通常演奏状態・テンポ変化状態 → 停止状態
拍が 3 回連続で検出されなかった場合に遷移する.

2.3 担当 C — 楽譜・演奏制御

2.3.1 機能一覧

担当 C では、以下の 2 点を担当する.

(1) 楽譜データの作成

(2) Arduino-Processing 間の通信システムの設計・実装

各楽器の演奏開始タイミングは、親機 Processing から送信される輝度キューによって決定される. 楽器の演奏予定順序を表 2.2 に示す. 最終的な演奏順序は発表当日に Processing の楽器指定ボタンから入力する.

表 2.2 演奏予定順序 (親機から動的に指示)

楽器	演奏開始	パート
ハイハット・シンバル	0 小節目	リズム 1
バス・スネア	0 小節目	リズム 2
鉄琴	2 小節目	主旋律 1
ピアノ	4 小節目	主旋律 2
トランペット	6 小節目	主旋律 3

演奏テンポは表 2.3 の通りである.

表 2.3 演奏テンポ一覧表

区間	テンポ
開始～14 小節（一周目演奏終了）	80bpm
15 小節目（二周目演奏開始）以降	104bpm

演奏用 Arduino-Processing 間の通信は有線接続によってシリアル通信を行う。
Arduino から Processing へ送信する内容一覧は表 2.4 に記す。

表 2.4 シリアル通信パケット仕様

バイト	フィールド名	内容	型	範囲
0	NOTE_OFF	前の音の終了	uint8	true = 1, false = 0
1	NOTE	音名インデックス	uint8	0～127 (255 = 休符)
2	VEL	音の強さ	uint8	0～127

2.3.2 システム構成図

担当 C のシステム構成図は図 2.6 の通りである。

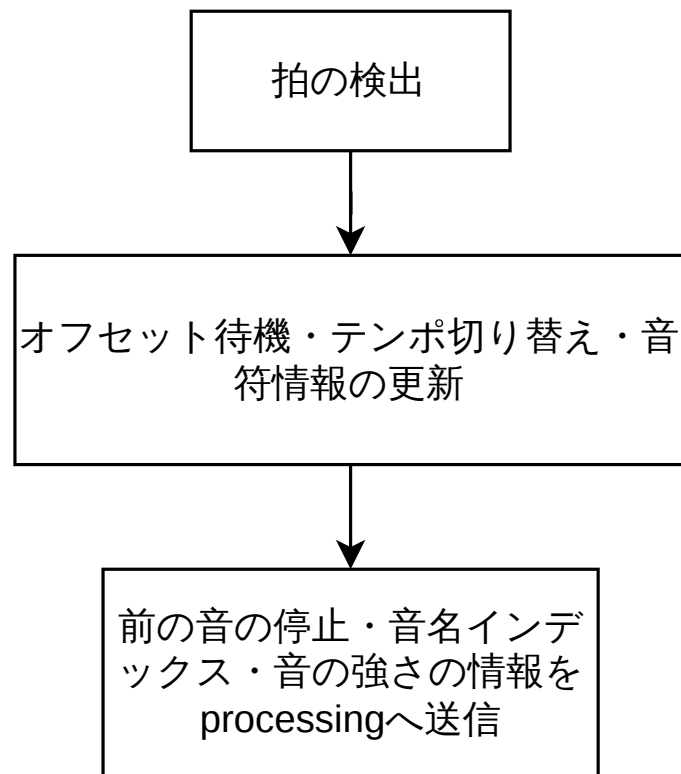


図 2.6 担当 C システム構成図

2.4 担当 D — 音響合成 (Processing/Minim)

2.4.1 機能一覧

本担当では, Arduino (担当 C) から送信されるシリアル通信データを受信し, Processing 上で音響合成を行い, 各楽器音をリアルタイムに出力する機能を実装する.

主な機能は以下の通りである.

- ・ シリアル通信受信機能

Arduino から送信される 3 バイトパケット (FLAG / NOTE / VEL) を受信し、NOTE_ON および NOTE_OFF イベントとして解釈する。ボーレートは 115200bps とする。

- ・ 受信データパース機能

受信したバイト列を 3 バイト単位で解析し、FLAG (0x01=NOTE_ON / 0x00=NOTE_OFF)、音名インデックス、およびベロシティ情報を抽出する。

- ・ 音高変換機能

音名インデックス (C4=0, D4=1, E4=2, F4=3, G4=4, A4=5, 休符=255) を MIDI ノート番号へ変換し、 $f = 440 \times 2^{(n-69)/12}$ [Hz] により周波数を算出する。変換テーブルは担当 C と合意のうえ実装する。

- ・ 音響合成機能

Minim ライブラリを用い、Oscil・ADSR・フィルタ・BitCrusher を直列接続した UGen チェーンにより各楽器音をリアルタイムに生成する。

- ・ 楽器音色生成機能

ハイハット・シンバル、バスドラム・スネア、鉄琴、ピアノ、トランペットの 5 パートそれぞれに対し、倍音比率・ADSR パラメータ・フィルタ設定を個別に定義し、識別可能な音色を実現する。

- ・ 複合音生成機能

ハイハット+シンバル、バスドラム+スネアのように、1 つの NOTE_ON イベントで複数の音源を同時に発音する。

- ・ 発音・消音制御機能

NOTE_ON 受信時に ADSR のアタックを開始し、NOTE_OFF 受信時にリリースへ移行する。フェルマータ中は NOTE_OFF が送信されないため、Processing 側は特別な処理を行わず Sustain 状態を維持する。腕再開後に担当 C から NOTE_OFF が送信された時点でリリースを開始し、続く NOTE_ON で次音を発音する。

- ・ 多重発音機能

テンポ変化時など前音のリリース中に次音の NOTE_ON が到達した場合、前音を強制消音せずに重ねて発音することで、音の途切れによる不自然さを回避する。

- ・ 操作インターフェース機能

ハッカソン本番での迅速なセットアップを支援するため、シリアルポートのド

ロップダウン選択 UI, マスターボリュームスライダー (0~100%), および NOTE_ON/OFF の受信パターンから推測可能なシステム状態のリアルタイム表示パネルを Processing 上に実装する.

- ・デバッグ・ログ出力機能

受信パケット (FLAG / NOTE / VEL) のパース結果および NOTE_ON/OFF イベントの発音タイミングをシリアルモニタへ出力し, 開発・動作確認を支援する.

2.4.2 システム構成図

本構成図では, 担当 D (Processing/Minim) の内部構成と, 上流の担当 C (Arduino 子機) および下流のスピーカー出力との接続関係を示す.

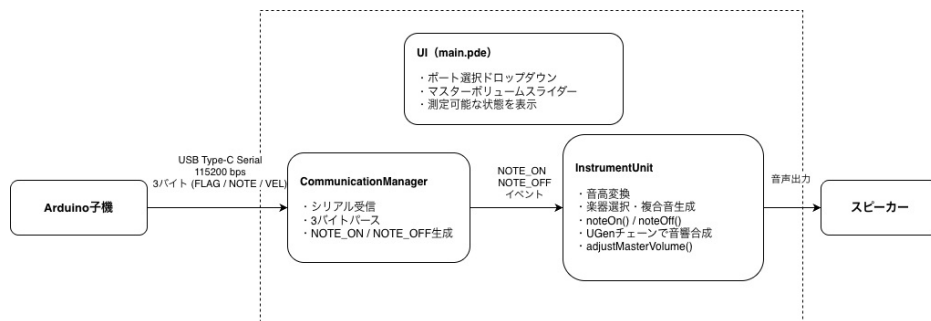


図 2.7 担当 D システム構成図

図に示す通り, 本システムは以下の 3 層で構成される.

通信層 — **CommunicationManager** Arduino 子機から USB Type-C Serial を介して 115200 bps で送信される 3 バイトパケット (FLAG / NOTE / VEL) を受信・解析し, NOTE_ON および NOTE_OFF イベントとして音響層へ通知する. また, シリアルポート選択 UI およびログ出力を管理する.

音響合成層 — **InstrumentUnit** 受信イベントに基づき, 音高変換, 楽器選択, 波形生成, および UGen チェーンによる音響合成処理を統括する. 各楽器パートに対応する InstrumentUnit インスタンスを独立して保持し, 複合音・多重発音にも対応する.

出力層 — **AudioOutputMinim** ライブラリの AudioOutput を通じてスピーカーへ音響信号を送出する. マスターボリュームスライダーの値 (0.0~1.0) を `out.setVolume()` へ渡すことでアンサンブル全体の音量を調整可能とする.

各層間のデータフローを以下に示す.

Arduino 子機 $\xrightarrow{\text{Serial 115200bps / 3B}}$ CommunicationManager $\xrightarrow{\text{NOTE_ON/OFF}}$ InstrumentUnit
 $\xrightarrow{\text{UGen チェーン}}$ AudioOutput $\rightarrow$ スピーカー

なお, UGen チェーンの詳細 (Oscil・ADSR・フィルタ・BitCrusher の接続構成) は次節 (音響合成チェーン) に示す.

2.4.3 音響合成チェーン

受信した NOTE 情報は, 直ちに以下の UGen チェーンに渡され, 各楽器音がリアルタイムに合成・出力される.

Oscil (基音+倍音) $\rightarrow$ ADSR $\rightarrow$ Filter (LowPassFS / HighPassFS) $\rightarrow$ BitCrusher
 $\rightarrow$ AudioOutput

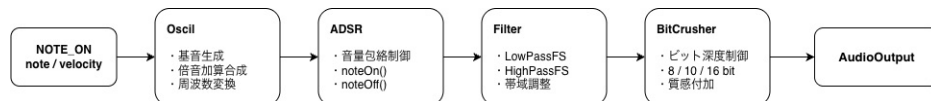


図 2.8 UGen 合成チェーン図

基音の生成から始まり, 倍音の付加, ADSR による音量エンベロープの適用, フィルタによる帯域調整, BitCrusher による質感付加を経て, 最終的なスピーカー出力となる. 各ユニットの詳細なパラメータ設定については詳細設計に示す.

2.4.4 操作インターフェース設計

ハッカソン本番の現場における迅速なセットアップと, 開発・デバッグ時の動作確認を目的として, Processing 上に以下のコントロールパネルを実装する.

2.4.4.1 ポート選択 UI と通信初期化

各 PC における Arduino の認識状況 (COM ポート番号) が異なることを想定し, シリアルポートをドロップダウン形式で動的に選択できる UI を配置する. 接続失敗時には警告メッセージを表示し, 現場での迅速なトラブルシューティングを支援する.

2.4.4.2 マスターボリューム・コントロール

5 台の PC から出力される音量を合奏形式で調整するため, 0~100% の範囲で操作可能なマスターボリュームスライダーを設ける. 各 PC の OS 音量設定に依存する

ことなく、アンサンブルの聴感上のバランスをリハーサル中に即座に最適化できる。

2.4.4.3 リアルタイム状態モニタ

開発・デバッグ時のシステム内部状態確認を目的として、NOTE_ON / NOTE_OFF の受信パターンから推測可能な状態を視覚的に表示するステータスパネルを実装する。

確定仕様の 3 バイトパケット (FLAG / NOTE / VEL) には担当 B の STATE 情報が含まれないため、Processing 側が自律的に判断できる状態を表 2.5 に示す。

表 2.5 状態モニタの表示状態一覧

状態名	表示方法	説明
IDLE	確実に判断可	NOTE_ON 未受信またはシリアル未接続
PLAYING	確実に判断可	NOTE_ON 受信後、発音中
FERMATA	推測 (目安 2 秒)	PLAYING 中に NOTE_OFF が 2 秒以上未受信の場合
CALIBRATE	表示不可 (—)	Arduino の内部状態のため受信不可
PREBEAT	表示不可 (—)	同上
TEMPO_CHANGE	表示不可 (—)	同上

なお、担当 C との合意により 3 バイトパケットに STATE 情報 (1 バイト) を追加する場合は、6 状態すべての正確な表示へ拡張することが可能である。

2.4.5 状態遷移図

本システムはイベント駆動型であり、Arduino からの NOTE_ON/OFF イベントをトリガとして動作する。担当 D の状態管理はシンプルな 2 状態で構成される。

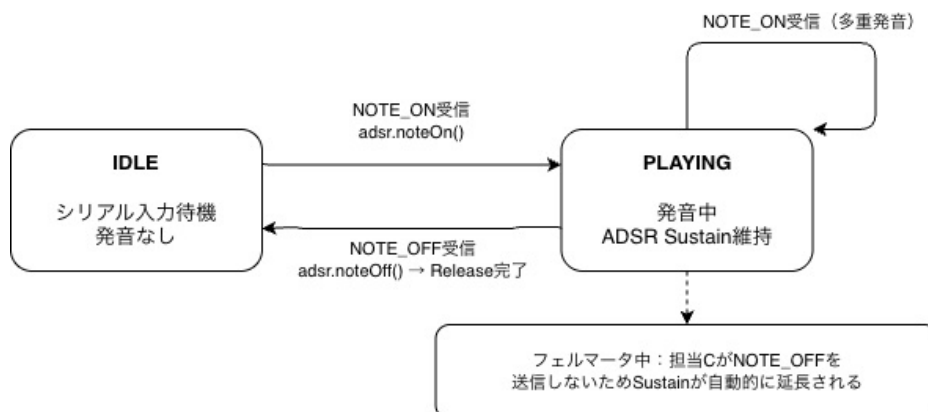


図 2.9 担当 D 状態遷移図

IDLE初期状態。シリアル入力待機中。NOTE_ON 受信により PLAYING 状態へ遷移する。

PLAYING発音中の状態。ADSR に従って音を維持する。NOTE_OFF 受信により Release フェーズを経て IDLE へ復帰する。フェルマータ中は担当 C が NOTE_OFF の送信を保留するため、Sustain が自動的に延長される。腕再開後に担当 C から NOTE_OFF が送信された時点で Release を開始する。

なお、複数パートの同時発音においては、各 InstrumentUnit インスタンスが独立した状態を保持するため、あるパートが PLAYING 中であっても他パートは独立して IDLE/PLAYING を遷移できる。

第 3 章 システムの詳細設計

設計書は作成する成果物によって変化します。適宜修正して利用してください。

3.1 担当 A — 指揮者人形・センサ機構

3.1.1 部品一覧

本システムの構築にあたって、使用を想定している主要部品を以下の表 3.1 に示す。

表 3.1 部品一覧

部品名	型番・仕様	備考
マイクロコントローラ	Arduino Uno R4 WiFi	システム全体の制御を担当
サーボモータ	SG92R	負荷に応じて金属製ギアのものに変更
反射板（腕）	プラスチックなど	軽量かつ赤外線を反射しやすい素材を選定
指揮用 LED	高輝度単色 LED（赤または白）	子機センサが検知しやすいものを選定
状態表示用 LCD	SSCI-014076	BPM や状態を表示

3.1.2 ハードウェア設計

3.1.2.1 ユニット構成

本ユニットにおける具体的な設計案を以下の図 3.1 に示す。

本ユニットは、制御・処理を担うマイクロコントローラ（Arduino Uno R4 WiFi）、腕の物理動作を行うサーボモータ、拍タイミングを光で提示する指揮用 LED、演奏状態を表示する LCD ディスプレイで構成される。

なお、赤外線測距センサおよびフォトランジスタは本ユニットには搭載しない。これらは各楽器ユニット（子機）側に搭載され、指揮者人形ユニット（親機）の腕

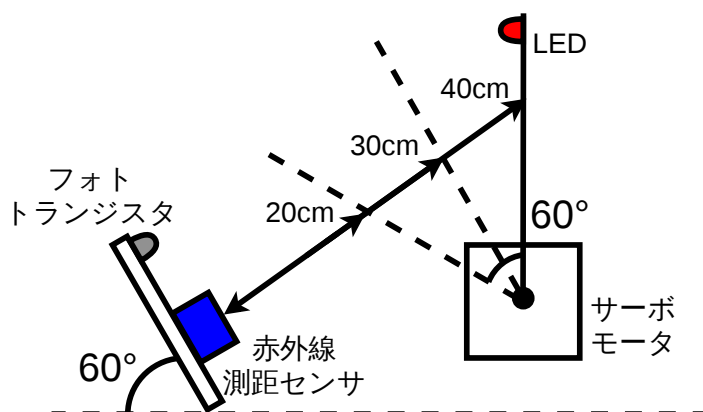


図 3.1 指揮者人形ユニットの設計図

(反射板) の位置変化および指揮用 LED の点灯を観測することで、通信に依存せず自律的に拍を同期する役割を担う。

3.1.2.2 ピン接続

Arduino Uno R4 WiFi と各デバイスのピン接続を以下の表 3.2 に示す。

表 3.2 ピン接続一覧

Arduino ピン	接続先デバイス	信号種別
D9 (PWM)	サーボモータ	PWM 出力
D6 (PWM)	指揮用 LED アノード (330 Ω 経由)	PWM 出力
SDA	LCD (SSCI-014076)	I ² C データ
SCL	LCD (SSCI-014076)	I ² C クロック
USB Type-C	PC (Processing)	シリアル通信 (115200bps)

3.1.2.3 回路設計

本ユニットにおける回路接続図を以下の図 3.2 に示す。

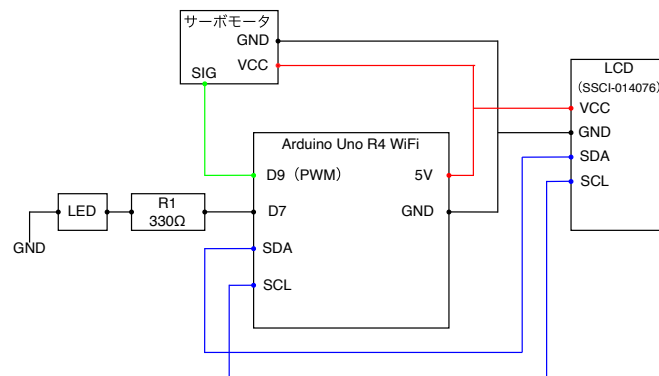


図 3.2 回路接続図

3.1.3 ソフトウェア設計

3.1.3.1 概要

本ユニットのソフトウェアは、Arduino Uno R4 WiFi 上で動作するファームウェアと、PC 上で動作する制御アプリケーション（Processing）の 2 つで構成される。

ファームウェアは、演奏の状態管理・サーボモータの制御・LED 点灯・LCD 表示・シリアル通信受信を担う。状態管理には「待機」「予備拍」「演奏中」「フェルマータ」「演奏停止」の 5 つの状態を定義し、PC からのコマンド受信をトリガとして状態を遷移させる。拍のタイミング生成には FspTimer による割り込みを用い、メインループに依存しないマイクロ秒単位の精度でサーボモータと LED を同期制御する。LED 制御は演奏フェーズによって 2 段階で切り替わる。演奏開始前の楽器指定フェーズでは PWM デューティ比を変調して輝度レベルを指定し、各子機へ演奏開始キューを送信する。演奏開始後はフル輝度（デューティ比 100%）の ON/OFF 制御に切り替え、拍信号として全子機へ送出する。

制御アプリケーション（Processing）は、楽器指定ボタン（楽器 1～5）、BPM 入力欄、START・STOP・フェルマータボタンを備えた専用 GUI インターフェースを提供する。ユーザの操作に応じて各コマンドをシリアル通信（115200bps）経由で Arduino へ送信する。

3.1.3.2 シリアル通信プロトコル

PC から Arduino へ送信するコマンドの仕様を表 3.3 に示す。

表 3.3 シリアル通信コマンド仕様

コマンド文字列	意味	遷移先状態
START	演奏開始	予備拍 → 演奏中
STOP	演奏停止	停止動作 → 待機
FERMATA	フェルマータ開始	フェルマータ
INSTRUMENT:n	楽器 n 番へキュー送信（輝度レベル変調）	演奏中（キュー状態）
BPM:nnn	BPM 設定（60～200）	即時反映

3.1.3.3 関数設計

Arduino のファームウェアおよび制御アプリケーション（Processing）それぞれの関数一覧を以下の表 3.4、表 3.5 に示す。

表 3.4 Arduino 側 関数一覧

関数名	引数	戻り値	役割
setup()	なし	なし	各デバイスの初期化、タイマ割り込みの設定
loop()	なし	なし	シリアル受信の監視、状態に応じた処理の呼び出し
onBeatTimer()	なし	なし	タイマ割り込み：サーボ角度更新・LED 点灯を同期実行
updateServo()	なし	なし	現在の状態・拍位相に基づきサーボ角度を更新
updateLED()	なし	なし	演奏開始前は PWM で輝度レベルを変調してキューを送信し、開始後はフル輝度 ON/OFF で拍信号を生成する
receiveCommand()	なし	なし	シリアルバッファを読み取りコマンド文字列を返す
changeState(State s)	遷移先状態 s	なし	状態変数を更新し、遷移時の初期化処理を行う
updateLCD()	なし	なし	現在の状態・BPM を LCD に表示する

表 3.5 Processing 側 関数一覧

関数名	引数	戻り値	役割
setup()	なし	なし	ウィンドウ・シリアルポートの初期化、UI ボタンの配置
draw()	なし	なし	UI の再描画（ボタン状態・接続状態の表示更新）
mousePressed()	なし	なし	クリック座標からボタンを判定し対応コマンドを送信
sendCommand(String cmd)	送信コマンド文字列 cmd	なし	シリアルポート経由で Arduino へコマンドを送信
drawButton(String label, int x, int y)	ラベル, X 座標, Y 座標	なし	指定位置にボタンを描画する
sendCue(int id)	楽器 ID (1～5)	なし	楽器 ID に対応する輝度レベルコマンドを Arduino へ送信する
setBPM(int bpm)	BPM 値 (60～200)	なし	BPM 値を Arduino へ送信する
drawInstrumentButtons()	なし	なし	楽器 1～5 の指定ボタンを描画する

制御アプリケーション（Processing）

3.1.4 処理フローの設計

メインループ処理およびタイマ割り込みの処理フロー図を以下の図 3.3, 図 3.4 に示す.

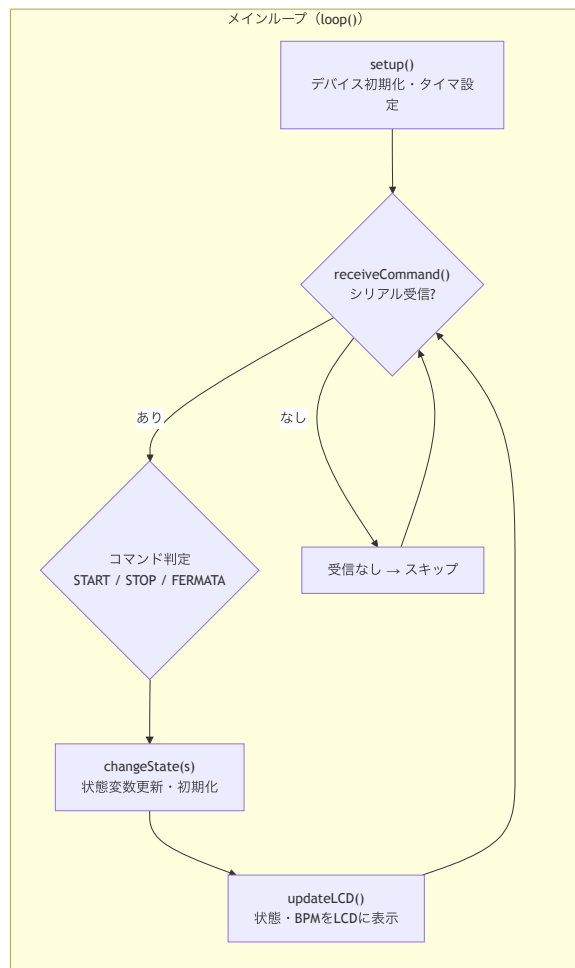


図 3.3 メインループ処理の処理フロー図

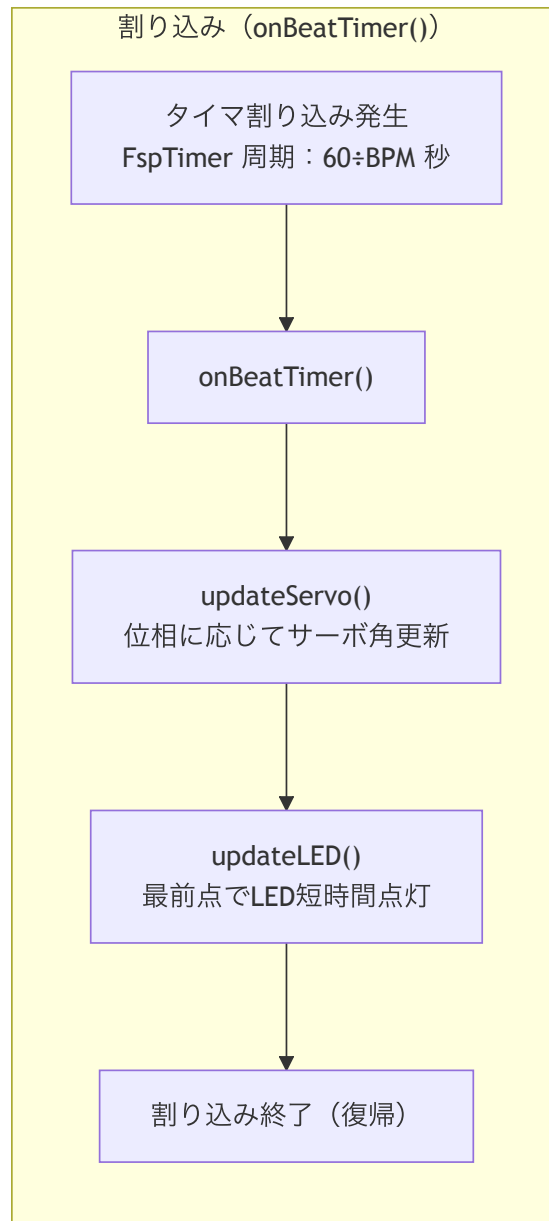


図 3.4 タイマ割り込みの処理フロー図

3.1.5 テストの設計

本ユニットの動作検証におけるテスト方針、および具体的なテスト項目について記述する。

3.1.5.1 テスト方針

本ユニットはシステム全体の親機として機能し、正確なテンポ維持と PC からのコマンドに応じた柔軟な状態遷移が求められる。そのため、テストは「ハードウェア単体テスト」「ソフトウェア機能テスト」「システム統合テスト」の3段階に分けて実施し、段階的に信頼性を確保する。

3.1.5.2 テスト項目一覧

具体的なテスト項目、検証内容、およびテストデータを表 3.6 に示す。

表 3.6 テスト項目一覧

カテゴリ	テスト項目名	検証内容	テストデータ・条件
ハードウェア単体	サーボモータ動作検証	指定した角度へ正確に追従し、腕（反射板）の負荷で脱調しないか	動作角度：0°～±30°
	指揮用 LED 視認性検証	LED が十分な光量で点灯し、子機側センサで検知可能か	点灯時間：最前点での短時間パルス点灯
	LCD 表示検証	動作状態や BPM が文字化けや遅延なく正しく表示されるか	各状態の文字列、BPM：60～140
ソフトウェア機能	シリアル通信・状態遷移	各コマンド受信により、定義された 5 つの状態へ正しく遷移するか	START, STOP, FERMATA、および予備拍・待機状態
	タイマ割り込み精度	FspTimer による割り込みがメインループに依存せず正確に作動するか	各種 BPM 設定時における割り込み周期の計測
システム統合	E2E フロー検証	Processing の UI 操作からコマンド送信、物理動作まで一連のフローが機能するか	各種ボタンのクリック操作（START → FERMATA → STOP など）
	BPM 境界値検証	高速・低速動作時にサーボの追従遅れやタイマの処理落ちがないか	最小設定 BPM、最大設定 BPM

3.2 担当 B — センサ処理・状態機械

担当 B は、子機 Arduino における指揮者人形の観測・テンポ推定・状態管理を担う。センサ構成は赤外線センサ（距離取得）とフォトトランジスタ（LED 検出）の2種類であり、両センサの情報を組み合わせてテンポ状態を判定し、発音タイミングを決定する。

3.2.1 部品一覧

表 3.7 使用部品一覧

部品名	型番	ピン	用途
赤外線センサ (×5)	シャープ測距モジュール GP2Y0A21YK	各楽器ユニットの A0	腕までの距離計測
フォトトランジスタ	照度センサ NJL7502L (560nm)	2 (割込), A1 (アナログ)	演奏開始前: 輝度キュー 判別 (analogRead), 演奏開始後: 拍タイミング検出 (デジタル割り込み)
Arduino (子機)	—	—	センサ処理・テンポ推定・状態管理

3.2.2 ハードウェア設計

赤外線センサ (GP2Y0A21YK) はアナログピン A0 に接続し, フォトトランジスタ (NJL7502L) はデジタルピン 2 (外部割り込み対応) に接続し, 演奏開始前の輝度判別時はアナログピン A1 でも読み取りを行う. 電源電圧は $V_{\text{ref}} = 5.0\text{V}$ とする.

赤外線センサの出力電圧 V から距離 d [cm] への変換は以下の近似式を用いる.

$$d = 27.728 \times V^{-1.2045} \quad (3.1)$$

フォトトランジスタは演奏開始前に `analogRead` で輝度レベルを読み取り, 自機 ID に対応するレベルを検出した時点でスタートフラグを立てる. 演奏開始後は `attachInterrupt` による外部割り込みに切り替えて拍検出時刻を取得する.

各楽器 ID と輝度レベルの対応は以下の通りである.

表 3.8 輝度レベルと楽器 ID の対応

楽器 ID	PWM デューティ比	analogRead 値（目安）
楽器 1	20%	≈ 200
楽器 2	40%	≈ 400
楽器 3	60%	≈ 600
楽器 4	80%	≈ 800
楽器 5	98%	≈ 1000
演奏中	100%（ON/OFF 制御）	拍信号

図 3.5 に回路図を示す.

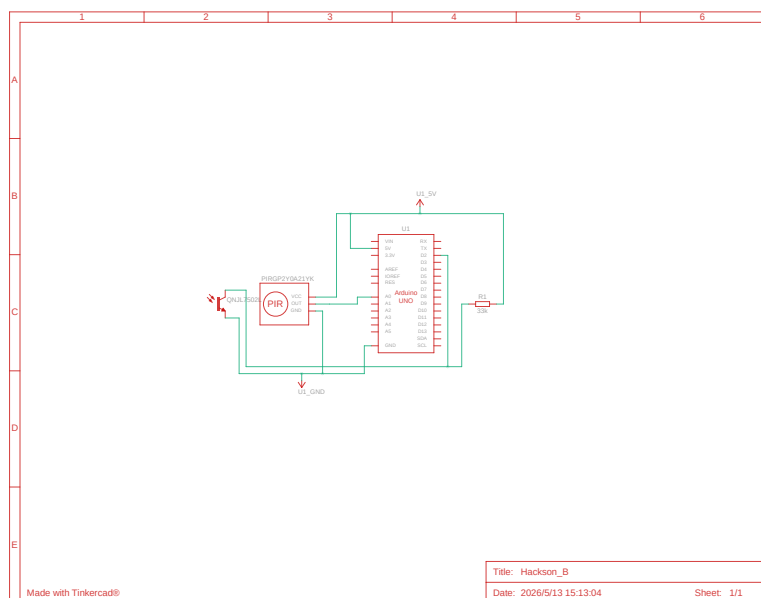


図 3.5 回路図

3.2.3 ソフトウェア設計

3.2.3.1 ファイル構成

表 3.9 ファイル構成

ファイル名	内容
hackson_main.ino	B と C の共通部分. setup() および loop() の エントリポイント, 共有変数の宣言を含む.
B.ino	担当 B の独立部分. センサ読み取り・テンポ推 定・状態機械の実装.
C.ino	担当 C の独立部分. 発音タイミング制御・信 号出力の実装.

3.2.3.2 オブジェクト設計 (クラス図)

本システムは Arduino 上で動作するため, クラスを用いず関数とグローバル変数で構成する. 主要変数を以下に示す.

Listing 3.1 担当 B 主要変数

```
1 // 赤外線センサ
2 volatile float irDistance = 0.0f; // 赤外線センサによる距離 [cm]
3 const int irWindowSize = 3;      // 移動平均のためのサンプル数
4 float irDistances[irWindowSize] = {0.0f}; // 距離バッファ (移動平均用)
5 float d_bar_n = 0.0f;    // 現在の距離移動平均 (正規化済み) [float
   d_bar_prev = 0.0f; // 前回の距離移動平均 (正規化済み) [
6
7 // フォトトランジスタ
8 bool beatDetected = false; // 拍検出フラグ (割り込みでセット)
9
10 // キャリブレーション
11 float irMax = 0.0f;    // キャリブレーション中の距離最大値 [cm]
12 float irMin = 9999.0f; // キャリブレーション中の距離最小値 [cm]
13
14 // EMA テンポ推定
15 float T_n = 500.0f;    // 現在の拍間隔推定値 [us]
16 float alpha = 0.2f;    // EMA 係数
```

```

17 unsigned long t_beat = 0;    // 現在の拍の検出時刻 [us]
18 unsigned long t_prev = 0;   // 前回の拍の検出時刻 [us]
19 unsigned long t_next = 0;    // 次回の推定拍検出時刻 [us]
20 const float TEMPO_CHANGE_THRESH = 10.0f; // テンポ変化判定閾値 [float
    tempoErrorRate = 0.0f; // テンポ誤差率 [bool isTempoChanged = false;
    // テンポ変化判定フラグ
21 bool beatUpdated = false;    // 推定拍更新フラグ（担当Cと共有）
22
23 // 赤外線センサによるテンポ推定
24 unsigned long tPredictBeat = 0; // 赤外線センサによる次の拍の推定絶対時刻
    [us]
25
26 // 状態機械
27 enum State {IDLE, CALIBRATE, PREBEAT, PLAYING, TEMPO_CHANGE, FERMATA};
28 State state = IDLE; // 現在の状態（担当Cと共有）
29
30 // 予備振り
31 int prebeatCount = 0;        // 予備振りカウント
32 const int PREBEAT_THRESH = 4; // 必要な予備振り回数
33
34 // テンポ変化からの通常演奏復帰
35 int stableCount = 0;        // 安定拍連続カウント
36 const int STABLE_THRESH = 3; // 復帰に必要な安定拍数
37
38 // 静止判定
39 const float STOP_THRESH = 5.0f; // 静止と判定される移動距離 [
40
41 // フェルマータ判定
42 int staticCount = 0;        // 静止サンプル連続カウント
43 const int FERMATA_THRESH = 3; // フェルマータ判定に必要な静止サンプル連続
    数
44
45 // フォールバック
46 int missedBeats = 0;        // 連続未検出拍数
47 const int MISS_THRESH = 3;   // 演奏中止に必要な未検出拍数
48
49 // その他
50 const float vRef = 5.0f;     // 電源電圧 [V]
51 #define IR_PIN A0            // 赤外線センサのピン
52 #define PHOTO_PIN 2          // フォトトランジスタのピン

```


3.2.3.3 関数設計

担当 B と担当 C は同一の Arduino 上で動作するため、以下のグローバル変数を共有することで連携する。担当 B が書き込み、担当 C が読み取る変数を以下に示す。

表 3.10 担当 C との共有変数一覧

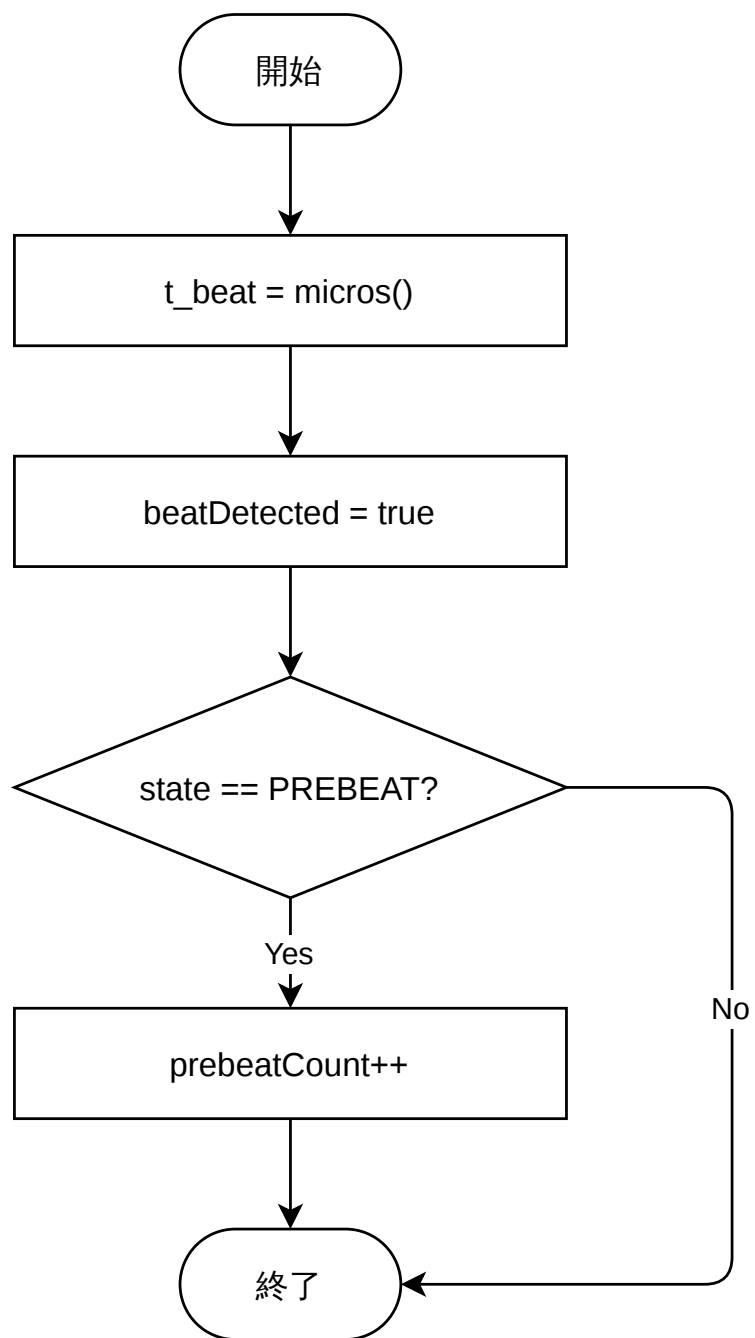
変数名	型	担当 B の役割	担当 C の使い方
t_next	unsigned	次の拍の予測絶対時	信号送信タイミング
	long	刻を書き込む	の基準として読む
beatUpdated	bool	t_next が更新された ら true にする	true を検知して処理 後 false に戻す
state	State	現在の演奏状態を書 き込む	FERMATA 判定などに 使う

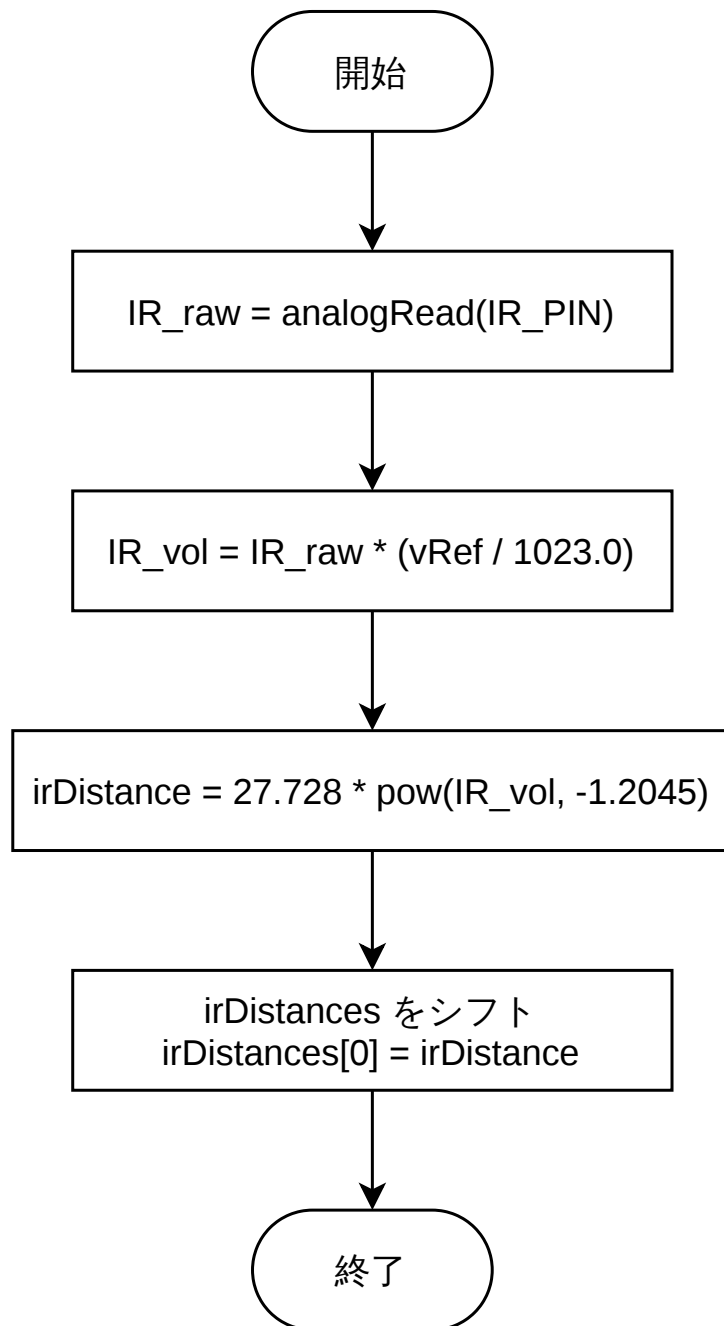
なお、beatUpdated を false にリセットする責任は担当 C 側にある。
主要関数の一覧を以下に示す。

表 3.11 担当 B 主要関数一覧

関数名	入力	出力	役割
void photoISR()	state	t_beat, beatDetected, prebeatCount	フォトランジスタ割り込み処理を行い、拍検出時刻と予備振り回数を更新する.
void readIR()	IR_PIN, vRef	irDistance, irDistances[]	赤外線センサから距離を取得し、移動平均用バッファを更新する.
void calibrate()	irDistance, irMax, irMin	irMax, irMin	キャリブレーション中の距離最大値・最小値を更新する.
void updateDBar()	irDistances[], irWindowSize, irMax, irMin	d_bar_prev, d_bar_n	距離データの移動平均を計算し、正規化距離を更新する.
void predictNextBeat()	未定	tPredictBeat	赤外線センサの動きから次回拍時刻を予測する.
void updateTempoByPhoto()	t_beat, t_prev, T_n, state	T_n, t_prev, t_next, isTempoChanged	フォトランジスタ検出結果をもとに EMA によりテンポ推定を更新する.
void updateTempoByIR()	t_next, tPredictBeat, T_n, state	T_n, t_next, isTempoChanged	赤外線センサによる拍予測値からテンポ変化を判定する.
bool detectFermata()	t_next, T_n, d_bar_n, d_bar_prev	staticCount, 戻り値	静止状態と時間超過条件からフェルマータ状態を判定する.
void updateState()	state, 各種判定結果	state, 各種状態変数	状態機械に基づいて状態遷移を管理する.

3.2.4 処理フローの設計





60
図 3.7 readIR

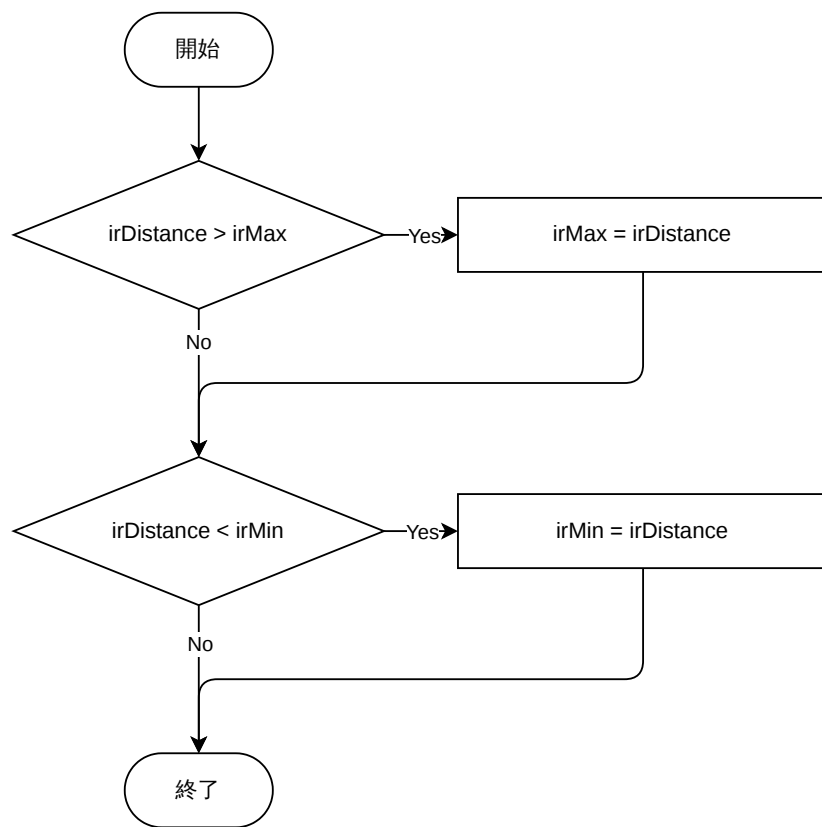
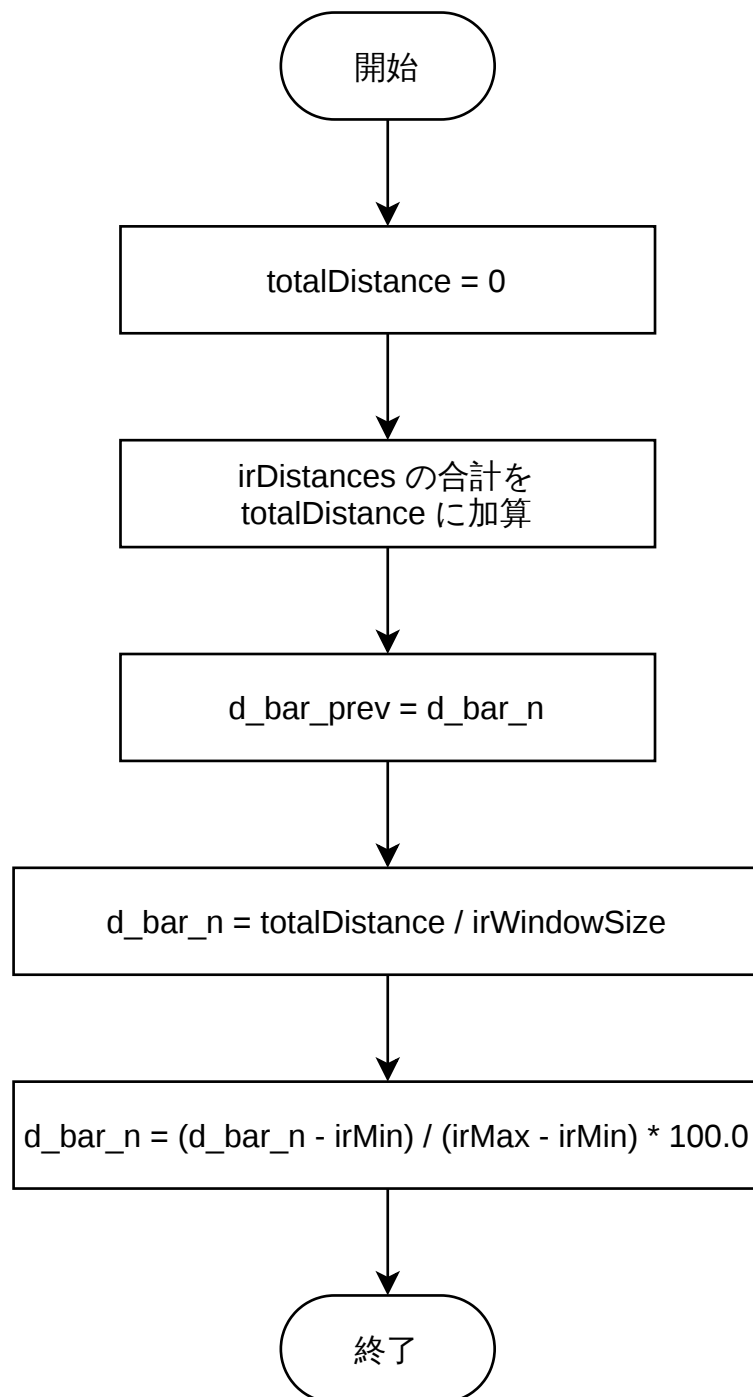


図 3.8 calibrate



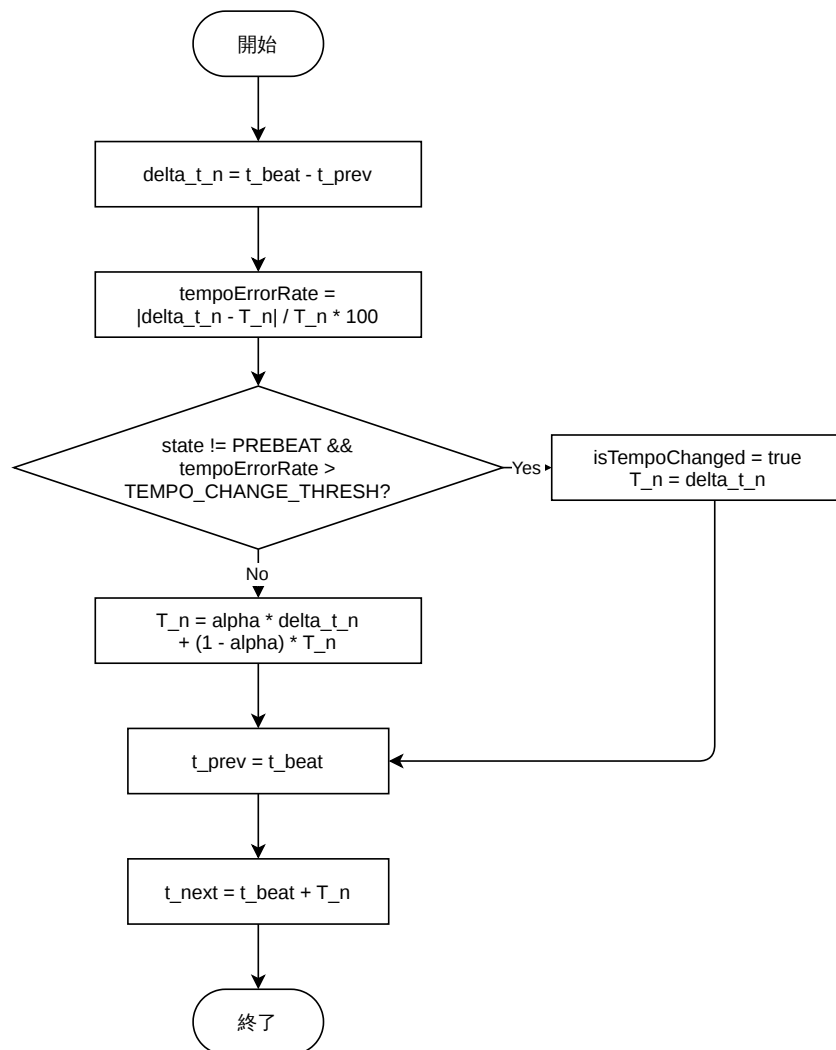


図 3.10 updateTempoByPhoto

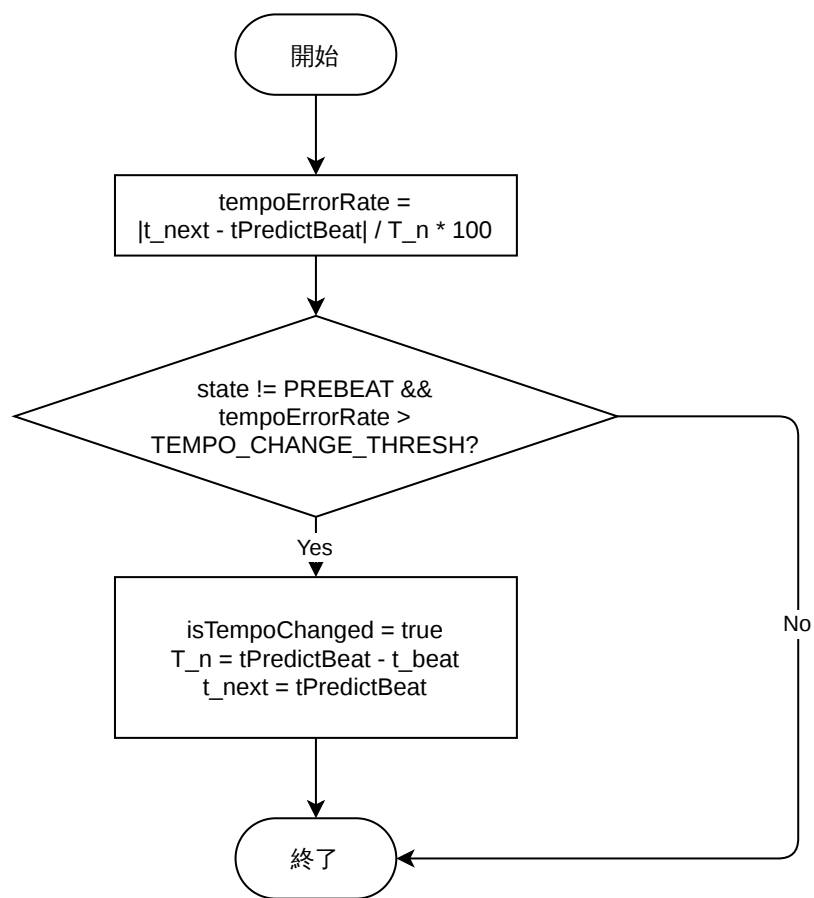


図 3.11 updateTempoByIR

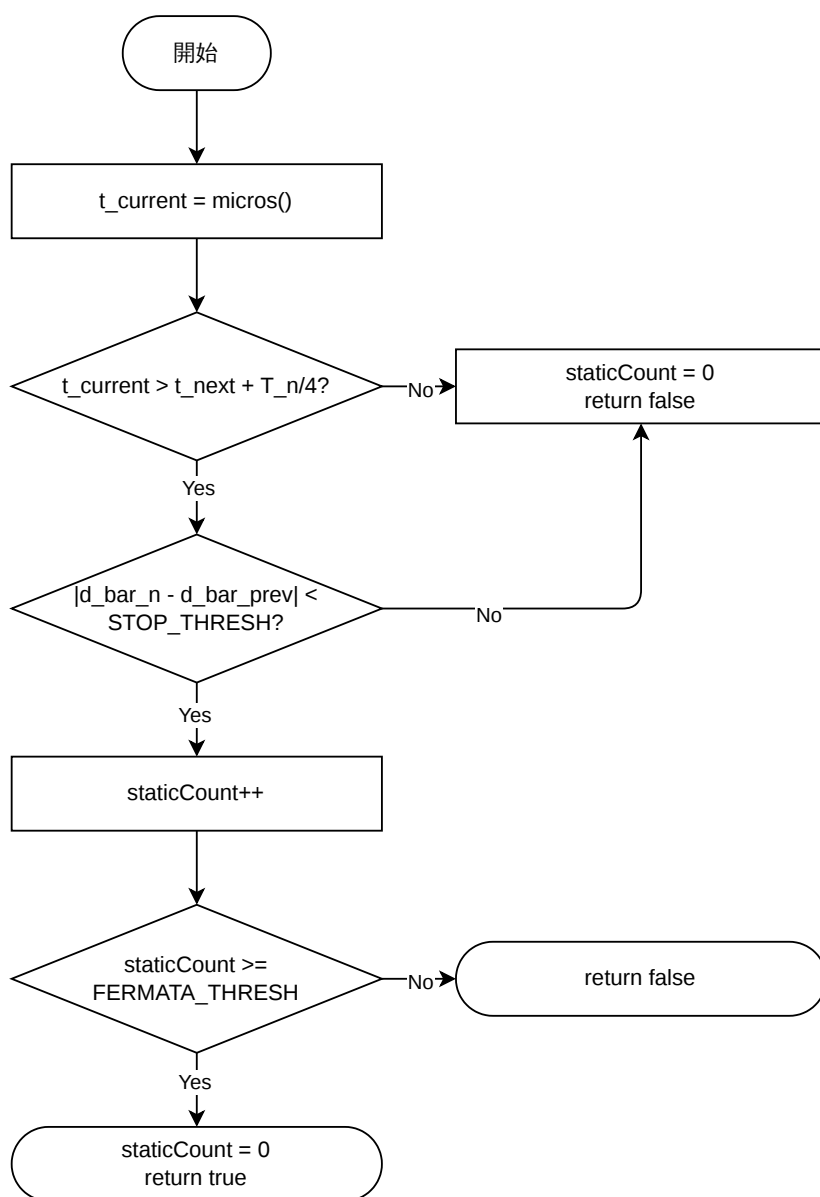


図 3.12 detectFermata

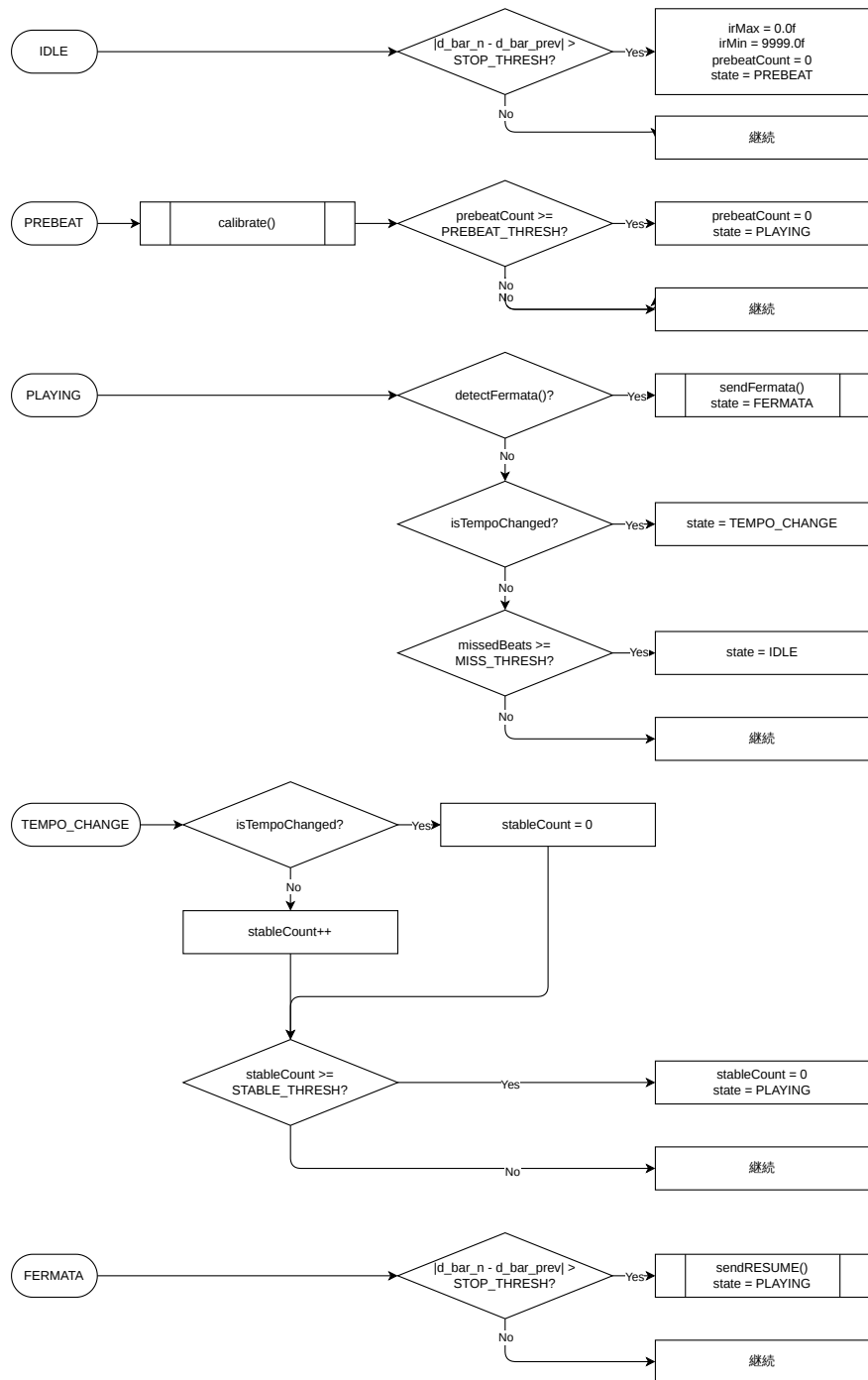


图 3.13 updateState

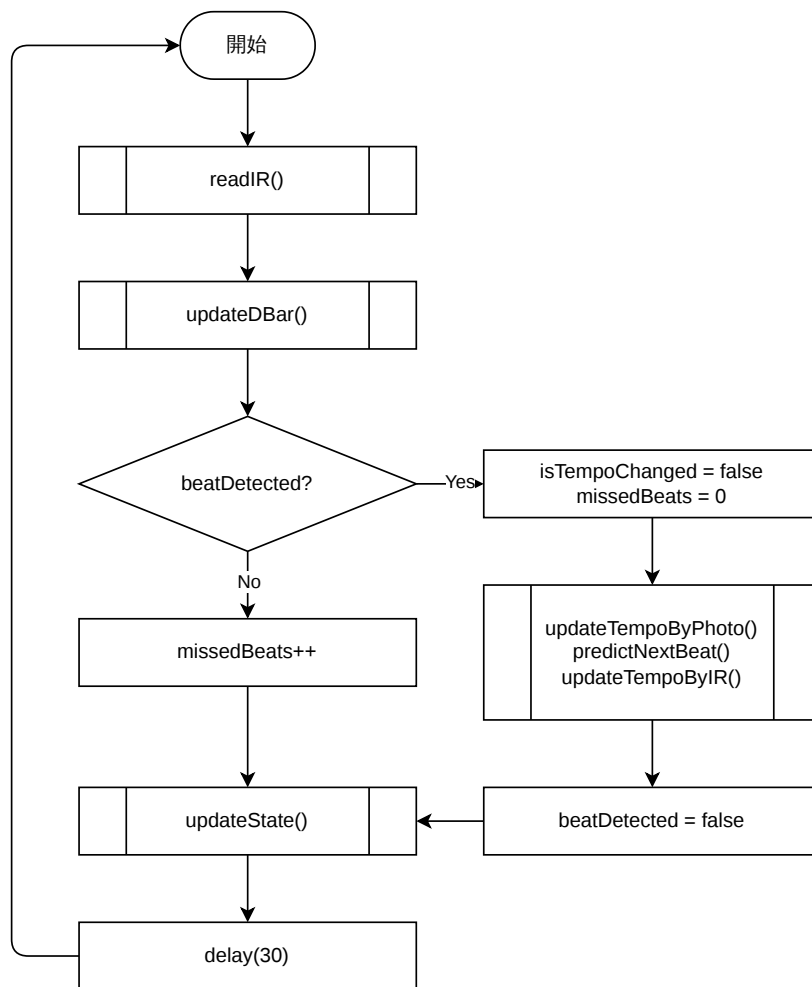


図 3.14 loop

3.2.5 テストの設計

担当 B が実施すべきテストの一覧を表 3.12 に示す.

表 3.12 テスト一覧

テスト名	内容	合格基準
測距センサキャリブレーション確認	既知距離で d_{norm} が 0～100%の範囲に正規化されるか確認する.	正規化誤差 $\pm 5\%$ 以内
センサ融合精度確認	メトロノーム (80bpm, 104bpm) に合わせて指揮動作を行い, 推定 BPM を計測する.	BPM 誤差 $\pm 5\text{bpm}$ 以内
EMA テンポ推定精度単体テスト	既知 BPM シーケンスをシリアル入力でシミュレートし, T_n の収束を確認する.	3 拍以内に BPM 誤差 $\pm 5\%$ に収束
状態機械遷移単体テスト	全 6 状態の遷移シーケンスをシリアルモニタでトレースする.	設計通りの遷移のみ発生, 無効遷移なし
フェルマータ検出単体テスト	腕停止→ FERMATA 遷移→ 腕再開→ PLAYING 復帰をシリアルで確認する.	停止後 2s 以内に検出, 誤検出なし
B-C 間遅延測定 (結合)	拍検出タイムスタンプと NOTE_ON 送信タイムスタンプの差をシリアルで計測する.	遅延 $\leq 50\text{ms}$
A-B 間テンポ追従確認 (結合)	人形 BPM を 80 → 104 に変化させ, 担当 B の EMA が追従するか確認する.	3 小節以内に新テンポに収束

3.3 担当 C — 楽譜・演奏制御

3.3.1 部品一覧

担当 C で必要となる器具と環境について表 3.13 に記す.

表 3.13 事前に揃えるべき器具・環境

品目	数量	用途
Arduino Uno R4 Wifi (子機用)	5	楽器ユニット各 1 台
USB Type-C ケーブル	5	各子機と PC 缶の Serial 通信
シリアル通信テスト環境 (Arduino IDE シリアルモニタ)	-	C1 テスト (B-C 間遅延測定)

3.3.2 ソフトウェア設計

3.3.2.1 楽譜データ構造

本システムで実装する楽譜データ構造は音名インデックス, 音の強さ, 拍数の 3 要素を持つ二次元配列として定義する. 音名インデックスの定義一覧は以下の通りである.

音名インデックス: $C4 = 0, D4 = 1, E4 = 2, F4 = 3, G4 = 4, A4 = 5$ (休符: 255)

定義例はソースコード 3.2 の通りである. 各パートの最後は休符を設定する.

ソースコード 3.2 楽譜配列（カエルの歌）

```
1  const uint8_t score[][3] = {
2      {0, 100, 1},    //ド
3      {1, 100, 1},    //レ
4      {2, 100, 1},    //ミ
5      {3, 100, 1},    //ファ
6      // ... 以下省略（前章節分を定義）
7  };
8
9  const int SCORE_LEN = sizeof ( score ) / sizeof ( score [0] ) ;
```

3.3.2.2 テンポ管理・演奏制御用の定数

各楽器の演奏開始は親機からの輝度キューによって決定される。子機は自機 ID に対応する輝度レベルを検出した時点でスタートフラグを立て、次の拍タイミングで演奏を開始する。演奏予定順序を参考として表 3.14 に示す。

表 3.14 演奏予定順序（親機から動的に指示）

楽器	演奏開始	パート
ハイハット・シンバル	0 小節目	リズム 1
バス・スネア	0 小節目	リズム 2
鉄琴	2 小節目	主旋律 1
ピアノ	4 小節目	主旋律 2
トランペット	6 小節目	主旋律 3

小節管理に使用する定数はソースコード 3.3 の通りである。

ソースコード 3.3 小節管理の定数

```
1
2  const uint8_t BEATS_PER_BAR = 4;  //小節あたりの拍数
```

演奏テンポ一覧は表 3.15 の通りである。

表 3.15 演奏テンポ一覧表

区間	テンポ
開始～14 小節（一周目演奏終了）	80bpm
15 小節目（二周目演奏開始）以降	104bpm

テンポ管理は小節カウンタ（ソースコード 3.3）による自立管理を行う。定義例はソースコード 3.4 の通りである。

ソースコード 3.4 テンポ切替定数

```

1
2  const uint16_t TEMPO_SLOW = 750;  //80bpm->拍間隔750ms
3
4  const uint16_t TEMPO_FAST = 577;  //104bpm->拍間隔577ms
5
6  const uint8_t TEMPO_CHANGE_BAR = 15;  //15小節目からTEMPO_FASTへ切替
7
8  uint16_t currentInterval = TEMPO_SLOW;  //現在の拍間隔[ms]
9  uint8_t barCount = 0;  //現在の小節カウンタ

```

3.3.2.3 オブジェクト設計（クラス図）

プログラム内で扱う変数一覧について表 3.16 に記す。

表 3.16 変数一覧

型	変数名	用途
int	noteIndex = 0	現在の音符ポインタ
int	beatCount = 0	経過拍数
int	remainBeats = 0	現在の音符の残り拍数
bool	playing = false	演奏フラグ（輝度キュー受信後に"true"になる）
bool	fermata = false	フェルマータ中フラグ

3.3.2.4 関数設計

本システムで実装する関数シグネチャと引数、戻り値、処理内容の一覧について表 3.17 に記す。特に複雑な関数（onBeatDetected(), advanceNote()）のフローチャートをそれぞれ図 3.15, 3.16 に示す。

表 3.17 関数一覧

関数名	引数	戻り値	処理内容
void onBeatDetected()	なし (void)	なし (void)	拍検出時の処理：輝度キュー受信によるスタートフラグ確認、テンポ切替、音符更新・送信、前の音の停止の送信を実行
void advanceNote()	なし (void)	なし (void)	残り拍数を減らし、0 になったら noteIndex を進める
void sendCurrentNote()	なし (void)	なし (void)	score[noteIndex] から NOTE ON パケットを生成し送信する
void sendNoteOn(uint8_t note, uint8_t vel)	uint8_t note : 音名インデックス (0 127 休符=255) uint8_t vel : 音の強さ (0 127)	なし (void)	演奏用 Arduino から Processing へシリアル送信する
void onFermata()	なし (void)	なし (void)	フェルマータ受信時、次の拍を検出するまでプログラムを停止する
void checkTempoChange()	なし (void)	なし (void)	barCount に応じて currentInterval を切り替える

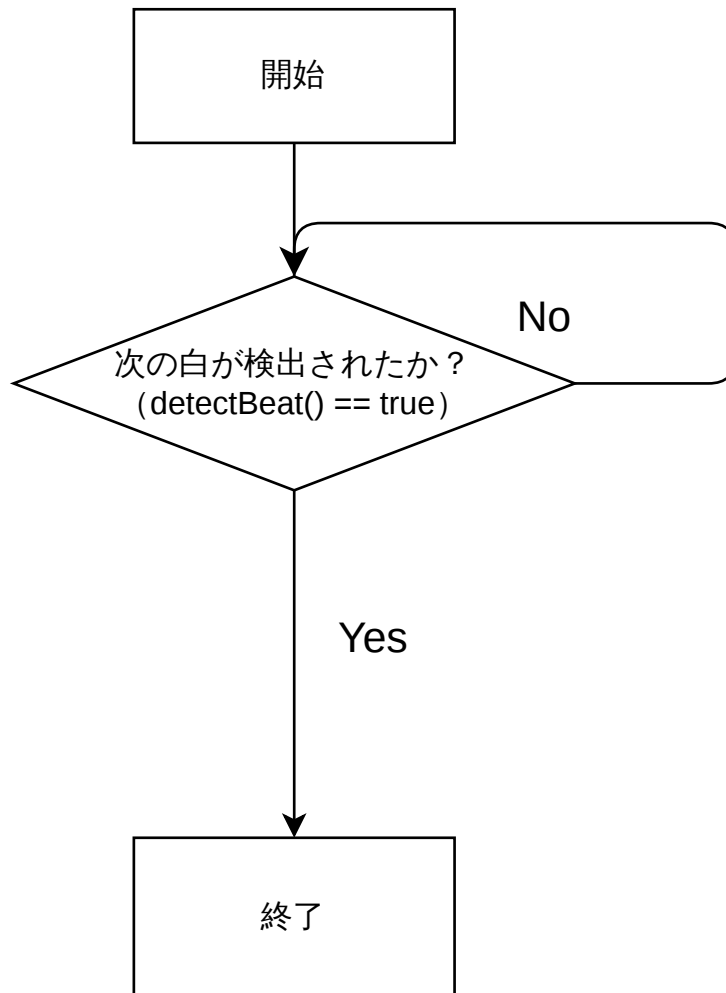


図 3.15 フローチャート : `onBeatDetected()`

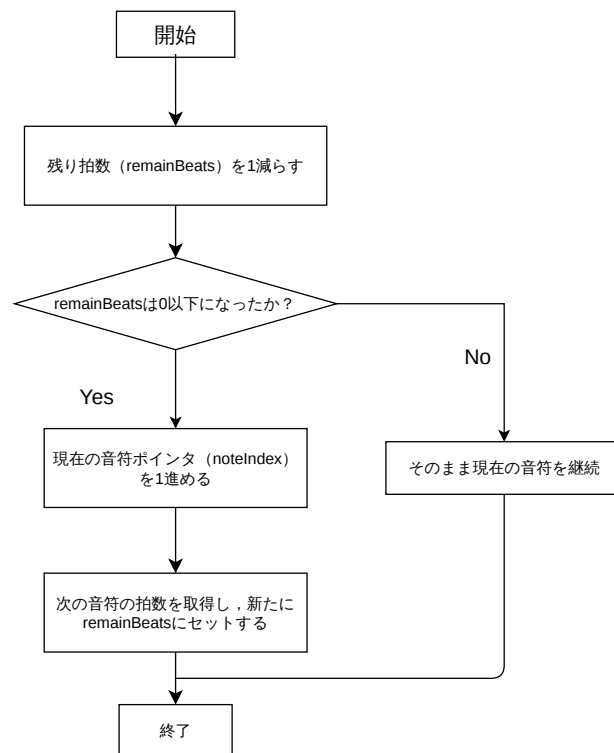


図 3.16 フローチャート：advanceNote()

3.3.3 テストの設計

本システムの課題として、シリアル通信時の遅延と音声出力時の遅延が考えられる。そのため、結合テストとして Arduino-Processing 間の往復遅延測定を行い、遅延補正值の測定・定数化を行う。

3.4 担当 D — 音響合成 (Processing/Minim)

3.4.1 部品一覧

本システムで使用する主な部品を以下に示す。本担当は 5 パート分のユニットを構成するため、以下の部品を各 5 台用意する。

表 3.18 担当 D 部品一覧

部品名	型番・仕様	数量	用途
PC (Processing 実行環境)	Processing 4.x + Minim 対応	5	音響合成・音声出力
スピーカーまたはヘッドフォン	—	5	音声再生
USB Type-C ケーブル	—	5	Arduino 子機との接続

3.4.2 ハードウェア設計

本担当では、PC 上での音響処理を主体とするため、専用の電子回路は使用しない。Arduino 子機（担当 C）から USB Type-C Serial を介してデータを受信し、PC 上の Processing/Minim により音響合成を行い、スピーカーへ音声出力する。

本担当は 5 パート分のユニットを独立して構成し、各ユニットは Arduino 子機 1 台・PC 1 台・スピーカー 1 台の組で動作する。

なお、各 PC は Arduino 子機 1 台と 1 対 1 で USB 接続されるため、受信した 3 バイトパケットはその PC が担当するパートの音響合成のみに使用する。

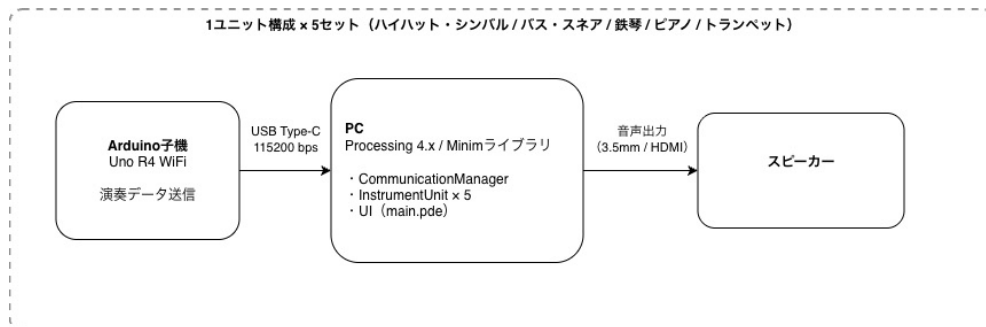


図 3.17 ハードウェア接続図（1 ユニット構成 × 5 セット）

115200 bps の USB シリアル通信で 3 バイトパケット（FLAG / NOTE / VEL）を受信する。

3.4.3 楽器音色設計の方針

聴衆が音楽として自然に認識できるよう、音響工学的な知見に基づき、各ユニットに明確な役割を持たせた音色設計を行う。

楽器グループの役割

- ・ピアノ・鉄琴・トランペット（主旋律担当）

倍音成分を調整し、メロディの明瞭さを追求する。基本振幅を高く設定し、常に聴衆の注意を引く音圧とする（基準音量 1.0 に対し 0.8～1.0）。

- ・バス・スネア（リズムの土台）

低域のエネルギーとアタックの鋭さで拍を明示する。主旋律を埋もれさせない範囲で、演奏の骨格を維持するために十分な音圧を確保する（基準音量 1.0 に対し 0.6～0.7）。

- ・ハイハット・シンバル（リズムのアクセント）

高域の金属音で演奏の華やかさを強調する。リズム楽器と同様の音量方針とする（基準音量 1.0 に対し 0.6～0.7）。

音色設計のアプローチ

楽器の音色をデジタル環境で再現するにあたり、以下のプロセスに基づいた設計を行う。

1. 物理特性分析：対象楽器の周波数成分を調和倍音の重なり（ピアノ・鉄琴・トランペット）と非調和・ノイズ成分（打楽器）に分類する。
2. 合成手法選定：加算合成による倍音構築か、ホワイトノイズをベースとした減算合成か、目的の音響モデルに応じて決定する。
3. 質感調整：ADSR エンベロープによる時間変化と、LowPass / BitCrusher による高域・ビット深度の制御を行い、聴覚的な違和感を排除する。

各パラメータの妥当性確認にあたっては、外部ツールを用いて既存音源と生成音のスペクトラムを比較し、基音および主要倍音の構成を確認しながら ADSR パラメータおよびフィルタ設定を逐次調整する。

3.4.4 音楽的表現の設計

指揮者のリアルタイムな動作に応じた変化が、人間らしい演奏として知覚されるための制御方針を以下に定める。

フェルマータの自然な表現

通信仕様を NOTE_ON / NOTE_OFF のトリガー方式としたことで、フェルマータの表現がより自然なものとなった。指揮者が静止した際、担当 C が NOTE_OFF の送信を遅らせることで、Processing 側は特別な処理を行わずとも ADSR の Sustain 状態が自動的に延長される。これにより、デジタル処理の不自然さを感じさせない、生身の演奏に近い「溜め」を実現する。

演奏再開時は、腕再開後に担当 C から NOTE_OFF が送信されることをトリガーとして Release を開始し、前の音の余韻を次音に滑らかに溶け込ませることで、デジタル特有の不自然な音切れを回避する。

テンポ変化時の自然な追従

テンポ変化に対し、デジタル的な違和感を排除するための配慮を組み込む。急なテンポアップ時に前の音が残っていても、消音せずに新しい音を重ねる多重発音を許容し、残響が途切れる不自然さを解消する。またテンポ加速時は Release 時間を短縮して歯切れを良くし、減速時は Release を延長して空白を響きで埋めることで、どの速度域でも自然な響きを維持する。

3.4.5 楽器別音響パラメータ

各楽器の ADSR 設定、倍音比率、および適用するエフェクトの数値を表 3.19 に示す。実装段階においては、アンサンブル全体のバランスや指揮への追従性を考慮し、聴感上の自然さを優先して逐次微調整する方針とする。

表 3.19 楽器別詳細パラメーター一覧

楽器名	波形構成・倍音比率	A [s]	D [s]	S [lvl]	R [s]	フィルタ	Bit
ピアノ	1.0, 0.6, 0.4, 0.3, 0.2, 0.1 (1~6 次)	0.01	0.15	0.2	1.00	2000 Hz (LP)	16
鉄琴	1.0, 0.4 (非調和), 0.1 (非調和)	0.01	0.05	0.1	1.20	8000 Hz (LP)	16
トランペット	1.0, 0.0, 0.7, 0.0, 0.5 (1, 3, 5 次)	0.10	0.05	0.8	0.20	3000 Hz (LP)	16
バス・スネア	Sine (低周波) + WhiteNoise	0.005	0.05	0.1	0.10	150 Hz (LP)	8
シンバル系	WhiteNoise 基準	0.005	0.005	0.1	0.50	9000 Hz (HP)	10

3.4.6 ソフトウェア設計

3.4.6.1 ファイル構成

本システムは Processing スケッチとして実装し、以下の構成とする。

- ・ main.pde

システム全体の制御を行う。初期化 (setup()), メインループ (draw()), および UI コンポーネント (ポート選択・マスターボリューム・状態モニタ) を含む。

- ・ CommunicationManager.pde

シリアル通信の受信および 3 バイトデータのパース処理を行う。NOTE_ON / NOTE_OFF イベントを生成し、InstrumentUnit へ通知する。各 PC は担当パートの InstrumentUnit1 インスタンスのみに通知する。

- ・ InstrumentUnit.pde

音響生成処理および楽器ごとの音色管理を行う。UGen チェーンの構築、音高変換、発音・消音制御を統括する。多重発音に対応するため、ADSR インスタンスを複数保持し、空きスロットを検索して発音するポリフォニック構成とする。

3.4.6.2 オブジェクト設計 (クラス図)

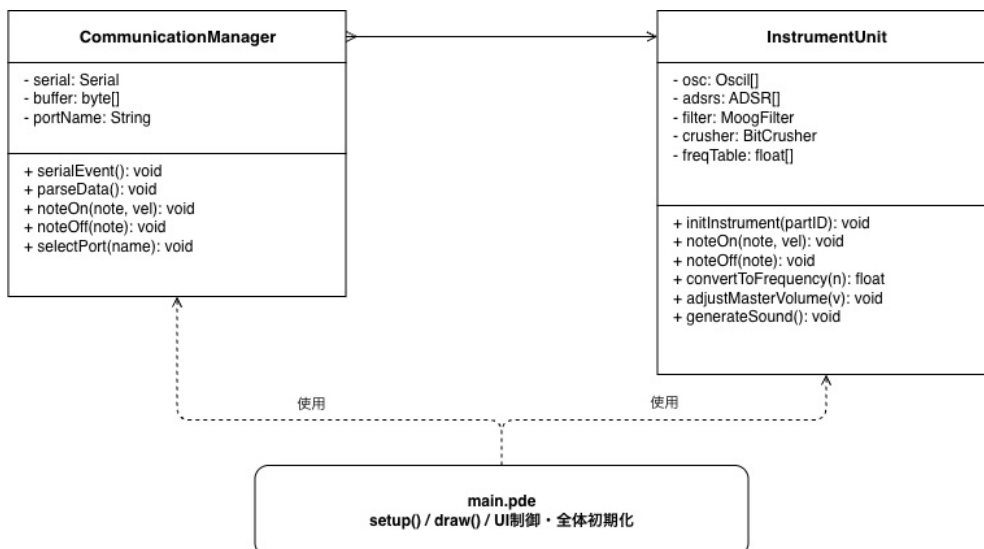


図 3.18 クラス図 (CommunicationManager / InstrumentUnit / main.pde)

本システムは以下の主要クラスで構成される。

- ・ `CommunicationManager`

シリアル通信の受信および 3 バイトパケット (FLAG / NOTE / VEL) のパースを行い、NOTE_ON / NOTE_OFF イベントを `InstrumentUnit` へ通知する。パート種別は各 PC のシリアル接続で確定するため、NOTE バイトはすべて音名インデックス (0~5 または 255) として解釈する。

- ・ `InstrumentUnit`

受信イベントに基づき、音高変換、楽器選択、複合音生成、および UGen チェーンによる音響生成処理を統括する。ADSR インスタンスを複数保持し、NOTE_ON 受信時に空きスロットを割り当て、NOTE_OFF 受信時に該当スロットを Release へ移行させることで多重発音 (ポリフォニック) を実現する。

音名インデックスからの周波数変換は、平均律式 $f = 440 \times 2^{(n-69)/12}$ [Hz] を用いる。インデックスと MIDI ノート番号の対応を表 3.20 に示す。

表 3.20 音名インデックス → MIDI ノート番号 → 周波数 変換テーブル			
インデックス (担当 C)	音名	MIDI ノート番号 n	周波数 f [Hz]
0	C4 (ド)	60	261.63
1	D4 (レ)	62	293.66
2	E4 (ミ)	64	329.63
3	F4 (ファ)	65	349.23
4	G4 (ソ)	67	392.00
5	A4 (ラ)	69 (基準)	440.00 (基準)
255	休符	—	発音なし

Velocity は 0~127 の値を ADSR の Sustain レベル (0.0~1.0) へ正規化し、音量制御に反映する。

NOTE_ON 時には `adsr.noteOn()`、NOTE_OFF 時には `adsr.noteOff()` を呼び出すことで音量包絡を制御する。フェルマータ中は NOTE_OFF が送信されないため、Sustain が自動的に延長される。複数の NOTE_ON を許容することで多重発音を実現する。

3.4.6.3 関数設計

主要な関数を以下に示す.

- `setup()`

Minim, AudioOutput, およびシリアル通信の初期化を行う. 各楽器パートの InstrumentUnit インスタンスを生成し, UGen チェーンを AudioOutput へ接続する.

- `draw()`

メインループとして動作し, UI コンポーネント (状態モニタ・マスターボリューム) の描画更新を行う.

- `serialEvent()`

シリアルデータ受信時に自動的に呼び出され, 受信バイトをバッファへ格納する.

- `parseData()`

受信バッファから 3 バイト単位でデータを取り出し, FLAG (0x01=NOTE_ON / 0x00=NOTE_OFF), 音名インデックス, Velocity を抽出して NOTE_ON / NOTE_OFF イベントに変換する. NOTE バイトはすべて音名インデックス (0~5 または 255) として解釈し, パート ID の抽出は行わない.

- `noteOn(int noteIndex, int velocity)`

音名インデックスを周波数に変換し, 空き ADSR スロットを選択して `adsr.noteOn()` を呼び出しアタックを開始する. Velocity を正規化して ADSR の Sustain レベルに反映する.

- `noteOff(int noteIndex)`

該当する音名インデックスの ADSR スロットを検索し, `adsr.noteOff()` を呼び出して Release フェーズへ移行させる.

- `convertToFrequency(int noteIndex)`

音名インデックスを MIDI ノート番号へ変換し, $f = 440 \times 2^{(n-69)/12}$ [Hz] により周波数を算出する.

- `initInstrument(int partID)`

指定パートの UGen チェーン (Oscil・ADSR・Filter・BitCrusher) を構築し, AudioOutput へ接続する. 楽器ごとの ADSR パラメータおよびフィルタ設定を適用する.

・ `adjustMasterVolume(float value)`

UI スライダーからの入力 (0.0~1.0) に基づき, `out.setVolume(value)` を呼び出して出力音量を調整する.

3.4.7 処理フローの設計

本システムの処理は, 初期化フローと実行時フローの2つに分けて設計する.

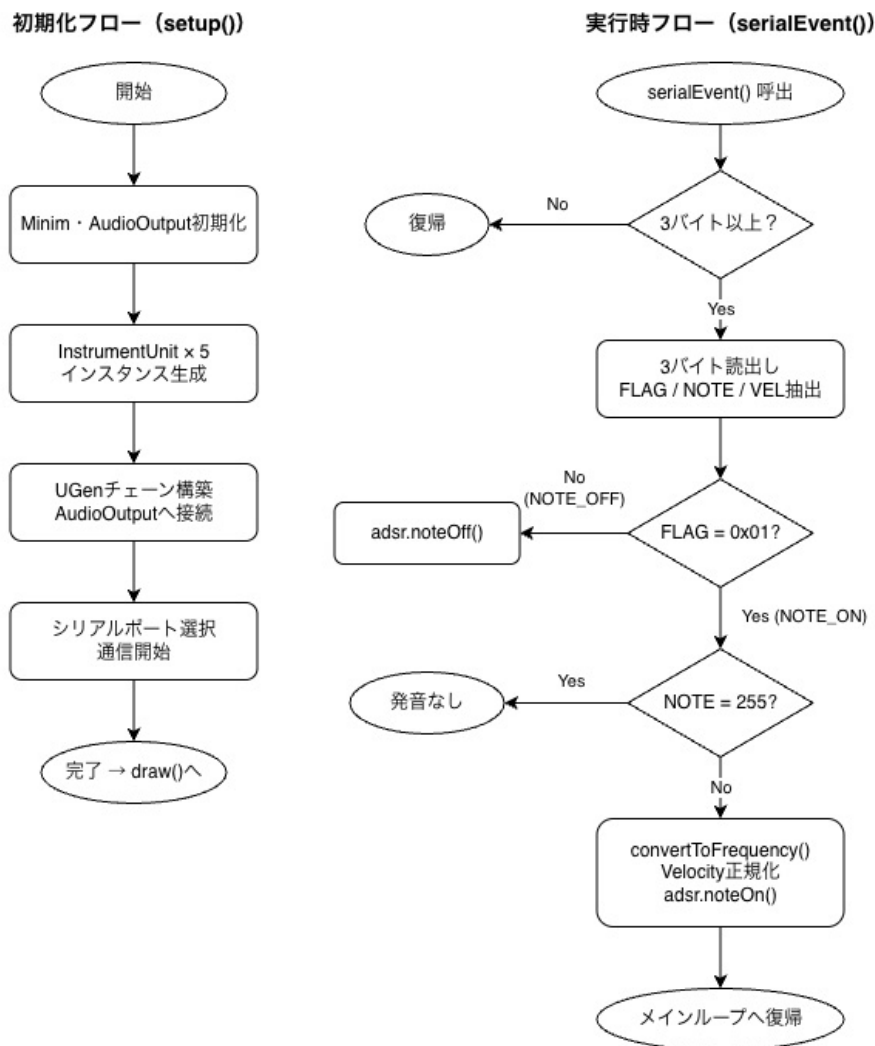


図 3.19 処理フロー図 (初期化フロー・実行時フロー)

初期化フロー (setup())

1. Minim およびシリアル通信を初期化する
2. 担当パートに対応する InstrumentUnit (1 インスタンス) を生成する
3. UGen チェーンを構築し, AudioOutput へ接続する
4. シリアルポートを選択し, 通信を開始する

実行時フロー (serialEvent() → parseData())

1. シリアルバッファに 3 バイト以上蓄積されているか確認する
2. 3 バイトを読み出し, FLAG・音名インデックス・Velocity を抽出する
3. FLAG を判定する
 - ・ FLAG = 0x01 (NOTE_ON) の場合:
 - (a) 音名インデックスが 255 (休符) であれば発音せずに終了する
 - (b) convertToFrequency() で周波数を算出する
 - (c) Velocity を正規化して ADSR の Sustain レベルに反映する
 - (d) 空き ADSR スロットを検索し, adsr.noteOn() を呼び出しアタックを開始する
 - ・ FLAG = 0x00 (NOTE_OFF) の場合:
 - (a) 音名インデックスに対応する ADSR スロットを検索し, adsr.noteOff() を呼び出し Release フェーズへ移行する
4. メインループへ復帰する

フェルマータ中は担当 C が NOTE_OFF の送信を保留するため, Processing 側は追加処理なく Sustain 状態が自動的に延長される. 腕再開後に NOTE_OFF を受信した時点で Release へ移行し, 続く NOTE_ON で次音の発音を開始する.

3.4.8 MOE / MOP / TPM

MOE — 有効指標

表 3.21 担当 D MOE 一覧

ID	指標	説明
MOE-D1	音楽的に正確な発音・消音	受信した NOTE 情報を正しい音高・音色・タイミングで発音・消音できる
MOE-D2	楽器らしい音色の実現	各楽器（ハイハット・バス・鉄琴・ピアノ・トランペット）が聴衆に識別できる音色を持つ

MOP — 性能指標

表 3.22 担当 D MOP 一覧

ID	性能指標	目標値	測定方法
MOP-D1	受信→発音遅延	≤ 50 ms	C2 テスト: NOTE_ON 受信タイムスタンプと Minim の発音開始を計測（可能であればオシロ）
MOP-D2	消音確実性	NOTE_OFF 受信後 ≤ 100 ms 以内に確実に消音	C2 テスト: 録音データで消音タイミングを確認
MOP-D3	音高精度	指定音名との誤差 ±5 cent 以内	チューナーアプリで各音名の発音周波数を計測
MOP-D4	Serial 受信エラーなし	通し演奏中に受信パースエラー 0 回	S1 テスト: エラーカウンタ変数をシリアルで監視

TPM — 技術性能指標（開発中に継続監視）

表 3.23 担当 D TPM 一覧

ID	監視項目	監視タイミング	判定基準
TPM-D1	3 バイト受信パーサの動作確認	修正後・P7 テスト時	FLAG=0x01 で NOTE_ON, 0x00 で NOTE_OFF が正しくトリガーされること
TPM-D2	音名インデックス→周波数変換の整合性	実装着手時 (C-D 合意後)	C4=0 のインデックスが $f = 261.6 \text{ Hz}$ に変換されること
TPM-D3	ADSR パラメータの音色調整状況	実装中・P7 テスト時	各楽器が識別可能な音色を持ち、音量バランスが取れていること
TPM-D4	Minim ライブラリ動作環境の統一	実装着手前	5 台の PC 全てで Processing + Minim が正常動作すること（バージョン統一）
TPM-D5	out.setVolume() の動作確認	P7 テスト時	スライダー操作により音量が 0~100% の範囲で連続的に変化すること

3.4.9 テストの設計

本システムのテストは、事前確認テスト・結合テスト・システムテストの 3 フェーズで実施する。なお、P7 が事前確認テスト、C2 が結合テスト、S1 がシステムテストにそれぞれ対応する。各テストの内容および合格基準を表 3.24 に示す。

表 3.24 担当 D テスト計画

ID	テスト名	内容	合格基準
P7	Processing 音響出力確認	手動で 3 バイト (0x01, NOTE, VEL) をシリアル送信し, 指定音が発音されるか確認する	正しい音高・音量で発音される
C2	C-D 間同期確認	Arduino の NOTE_ON 送信から発音までの遅延を計測する. また NOTE_OFF 受信時に確実に消音されるか確認する	遅延 ≤ 50 ms, NOTE_OFF 時に確実に消音, 発音ずれ ≤ 1 拍
S1	通し演奏・BPM 維持評価	全 15 小節以上の通し演奏で音響品質・タイミングを評価する	BPM 誤差 ± 5 bpm 以内, 発音ミスなし

また, 開発中の音色調整を目的として, 以下の評価を随時実施する.

- ・音高精度確認

チューナーアプリで各音名の発音周波数を計測し, 目標値との誤差が ± 5 cent 以内であることを確認する.

- ・聴感評価

各楽器音が聴衆に識別可能な音色を持つことを人間の聴感により確認する.

参考文献

- [1] Borchers, J., Lee, E., Samming, W., & Mühlhäuser, M. (2004). Personal orchestra: a real-time audio/video system for interactive conducting. *Multimedia Systems*, 9(5), 458–465.
- [2] Cont, A. (2008). ANTESCOFO: Anticipatory synchronization and control of interactive parameters in computer music. In *Proceedings of the International Computer Music Conference (ICMC)*, Belfast, Ireland.

- [3] 須藤宏明, 他, 「複数演奏者による音楽リズムタイミングの一致度の測定と脳波への影響の考察」, 日本感性工学会研究論文集, Vol.7, No.2, pp.337-344, 2007 年.
- [4] 河瀬諭, 「合奏における演奏者間コミュニケーション—タイミング調整と手がかり—」, 心理学研究, Vol.57, No.4, pp.495-, 日本心理学会.
- [5] 「短音の提示間隔と周波数および音圧の JND」, Audiology Japan, Vol.17, No.5, pp.399-400, 1974 年.